DRAGON

Divergence-Regulation for Adaptive Godunov-Oriented Numerics

Literary Overview & Reference Entries (LORE) Document

Bobbie Markwick

3 Nov 2026

Contents

1 Code Structure **1**
- 1.1 Directory Layout . . . 1
- 1.2 Data Structures . . . 2
  - 1.2.1 Fluid Elements . . . 2
  - 1.2.2 ExtendedArray . . . 3
  - 1.2.3 Grid . . . 3
- 1.3 Dependencies . . . 3
- 1.4 Style Guide . . . 4
  - 1.4.1 File Headers . . . 4
  - 1.4.2 Header guards . . . 4
  - 1.4.3 Includes . . . 4
  - 1.4.4 Naming . . . 5
  - 1.4.5 Indentation . . . 6
  - 1.4.6 Braces . . . 6
  - 1.4.7 Spaces . . . 6

2 Physical Model & Numerical Methods **7**
- 2.1 Governing Equations . . . 7
- 2.2 Initial Conditions . . . 7
- 2.3 Boundary Conditions . . . 8
  - 2.3.1 Outflow . . . 9
  - 2.3.2 Periodic . . . 9
  - 2.3.3 Reflective . . . 10
  - 2.3.4 Fixed . . . 10
  - 2.3.5 Ignore . . . 10
- 2.4 Multidimensional Integration Schemes . . . 10
  - 2.4.1 Operator-Split (Strang) Update . . . 10
  - 2.4.2 Unsplit (CTU) Update . . . 11
- 2.5 Time Integration . . . 12
  - 2.5.1 Per-Cell Signal Speed . . . 12
  - 2.5.2 Grid-Wide Timestep . . . 12
- 2.6 Reconstruction . . . 13
  - 2.6.1 Piecewise-Linear Slopes . . . 13
  - 2.6.2 MUSCL-Hancock Time Predictor . . . 14
  - 2.6.3 MHD Source terms . . . 14
  - 2.6.4 CTU Transverse Correction . . . 15
- 2.7 Riemann Solvers . . . 16

2.7.1 Exact (Hydrodynamics Only) . . . 16
2.7.2 HLL . . . 17
2.7.3 HLLE . . . 18
2.7.4 HLLC (Hydrodynamics Only) . . . 18
2.7.5 HLLD (MHD Only) . . . 18
2.7.6 Roe (Hydrodynamics Only) . . . 20
2.7.7 Riemann Solver Fallback Behaviour . . . 21
2.8 Conservative Updates to Primitive States . . . 22
2.9 Constrained Transport . . . 22
2.9.1 Computation of Electric Fields . . . 22
2.9.2 Faraday's Law . . . 23
2.9.3 Calculation of Body-Centred Fields . . . 24
2.10 Passive Scalars . . . 24

3 Source Terms **26**

4 Performance: DRAGONWING **27**
4.1 Memory Management . . . 27
4.2 Multithreading . . . 28
4.2.1 Checkpoint Protocol . . . 28
4.3 Domain Decomposition . . . 29
4.3.1 Load Balancing . . . 29
4.3.2 Ghost Cells . . . 30
4.3.3 Parallel Step & Synchronization . . . 30
4.4 Atomics . . . 31

5 File Output: DRAGONHOARD **33**
5.1 File Format . . . 33
5.2 Restart and Loading . . . 34

6 Running DRAGON **35**
6.1 Configuration . . . 35
6.2 Setting Up a Problem . . . 35
6.3 Building on macOS with Xcode . . . 36
6.4 Building from Command Line . . . 36
6.4.1 Creating the bin Directory . . . 36
6.4.2 Building on macOS . . . 36
6.4.3 Building on Linux . . . 37

7 Making Plots: DRAGONGAZE (Alpha) **38**
7.1 Setup . . . 38
7.2 Generating Plots . . . 38
7.3 Customizing Plot Appearance . . . 39
7.4 Additional Information for Developers . . . 39

8 Validation and Test Cases **40**
8.1 Unit Test Suite . . . 40
8.1.1 Component Tests . . . 40
8.1.2 Godunov Tests . . . 41
8.2 Example Problems . . . 41
8.2.1 Examples in One Dimension . . . 42
8.2.2 Hydrodynamic Examples in Two and Three dimensions . . . 43
8.2.3 MHD Examples in Two and Three dimensions . . . 46

A Additional Test Problem Plots **52**

B Version History **55**

Chapter 1

Code Structure

1.1 Directory Layout

```
DRAGON/                    # repository root
|-- DRAGON/                # core solver source
|   |-- Boundary/          # boundary conditions
|   |-- FluidElement/      # primitive/conservative state & arithmetic
|   |-- Hydro/             # hydrodynamic solver
|   |   |-- CFL/           # timestep control
|   |   |-- ExtendedArray/ # multidimensional arrays with ghost cells
|   |   |-- Godunov/       # split/unsplit driver, flux application
|   |   |-- Reconstruction/# MUSCL interface-state reconstruction
|   |   |-- Riemann/       # Exact, HLL, HLLE, HLLC, HLLD, Roe solvers
|   |-- MHD/               # constrained transport (E-field, Faraday, body fields)
|   |-- Passives/          # passive scalars
|   |-- Refinement/        # domain decomposition (DistGrid)
|   |-- main/              # program entry point & problem setup
|   |-- Config.h, Constants.h, Problem.cpp
|-- DRAGONWING/            # memory management & multithreading
|   |-- Atomics/           # atomic counters
|   |-- Memory/            # RAII scratch arrrays
|   |-- Multithreading/    # multithreading
|-- DRAGONHOARD/           # HDF5 output & restart I/O
|-- DRAGONGAZE/            # Python visualisation/analysis scripts
|-- DRAGONLORE/            # this documentation
|-- Examples/              # example problem setups
|   |-- 1D/                # shock tubes
|   |-- Hydro/             # 2D and 3D Hydro examples
|   |   |-- Blast/, Diagonal/, Kelvin-Helmholtz/
|   |-- MHD/               # 2D and 3D MHD examples
|   |   |-- Blast/, LoopAdvection/, MHDRotor/, Orszag-Tang/, Waves/
|-- Testing/               # unit tests
|   |-- Core/
|-- LICENSE                #Apache License
|-- README.md
```

Listing 1.1: Repository layout

1.2 Data Structures

1.2.1 Fluid Elements

The following data types are defined in `DRAGON/FluidElement/FluidElement.h`

vec3

`DRAGON::vec3` is a plain structure with x, y, z components. The following operations are supported:

- Addition (`+`, `+=`) and Subtraction (`-`, `-=`) with another `vec3`.
- Multiplication (`*`, `*=`) and Division (`/`, `/=`) by a scalar `double`.
- Dot (`*`) and cross (`cross(v1, v2)`) product between two `vec3`'s.
- Swapping two components, either in place (e.g. `swapXY()`) or on a copy (e.g. `swappedXY()`).

PrimitiveState

`DRAGON::PrimitiveState` represents a single fluid element (i.e. one cell's state), which contains density `rho`, pressure `p`, velocity vector `v`, and (MHD only) magnetic field vector `B`. This is the form the solver reconstructs, applies boundary conditions to, and reports as output.

As already noted in Chapter 2, `B` is stored in Gaussian (non-rationalized) units, so the code is explicit about factors of 4π wherever they appear – concretely, `Constants.h` predefines `_1_4pi` ($= 1/4\pi$), `_1_8pi` ($= 1/8\pi$), and `sq4pi` ($= \sqrt{4\pi}$) rather than dividing by 4π inline.

ConservativeState

`DRAGON::ConservativeState` represents a fluid element in conservative form: mass density `rho`, energy density `E`, momentum density vector `mom`, and (MHD only) `B`. This is the form fluxes are computed and applied in: rather than a true fluid element, a particular instance of `ConservativeState` might instead represent a flux (times one computational unit time per computational unit length).

The following fluid arithmetic operations are supported by `ConservativeState`

- Addition (`+`, `+=`) and Subtraction (`-`, `-=`) with another `ConservativeState`.
- Multiplication (`*`, `*=`) and Division (`/`, `/=`) of all components by a `double`.
- Swapping two dimensions as with `vec3`. This swaps the corresponding components of both `mom` and `B`. `PrimitiveState` also supports this operation directly (using `v` instead of `mom`).

Both `PrimitiveState` and `ConservativeState` are constructible from the other (`ConservativeState(PrimitiveState)` and `PrimitiveState(ConservativeState)`). This can be done explicitly, or implicitly whenever a `PrimitiveState` is used in fluid arithmetic[1]. If you need to directly add or subtract the components of two `PrimitiveState`'s, you will need to do this manually on each component.

Both types also provide a number of other physically relevant quantities, such as various wave speeds, `energy()` or `pressure()` on a primitive or conservative state, and `flux()` to calculate the fluxes of the governing MHD equations (2.1). The `flux()` function returns the flux in the x-direction, and in MHD includes the electric fields from the induction equation as the `B` components (see Section 2.9). When called from a `ConservativeState`, `flux()` can optionally take the velocity `v` if already known; otherwise `v` is computed from `mom` and `rho`.

[1] The `PrimitiveState += ConservativeState` operator has a special implementation for improved stability and performance (Section 2.8), but is otherwise equivalent to an implicit round-trip conversion.

1.2.2 ExtendedArray

Grid storage is built on a small family of templates in `DRAGON/Hydro/ExtendedArray`: `ExtendedArray1D<T>`, `ExtendedArray2D<T>`, and `ExtendedArray3D<T>`. Each owns a single flat, row-major heap array sized to include a configurable ghost margin. Accessing elements within an extended array is done via `array[i]`, `array[i,j]`, or `array[i,j,k]` depending on the array's dimension[2]. `i`, `j`, or `k` may be negative or exceed that dimension's size by an amount up to the array's ghost margin; in this case the accessed quantity corresponds not to a physical item but to cell within the boundary layer[3]. When DRAGON is built for testing, every cell access will assert that the bounds of the ghosts are within the valid range; this check is compiled out in production builds.

`ExtendedArray` objects can't be copied using normal C++ syntax, but the contents of one array can be copied to another (with identical dimension) using `clone(ref, copyghosts)`. Ghost cells are copied only if `copyghosts` is true.

`ArrayTypes.hpp` defines the concrete array types used throughout the solver as typedefs over these templates and `PrimitiveState`/`ConservativeState`/`vec3` (Section 1.2.1) – e.g. `FluidArray2D = ExtendedArray2D<PrimitiveState>`, `FluxArray3D = ExtendedArray3D<ConservativeState>`, `MagneticArray2D = ExtendedArray2D<vec3>`.

1.2.3 Grid

Fundamentally, a `DRAGON::Grid` (`DRAGON/Hydro/Grid.hpp`) is one or more extended arrays together with a boundary condition and the ability to advance the arrays in time.

The abstract `Grid` base class declares the two pure-virtual per-step advancement functions `split_step(dt)`/`unsplit_step(dt)`, the CFL-aware driver `advance(dt, check_cfl)` which wraps either `advance_split`/`advance_unsplit` (Section 2.5), and the virtual failure hook `on_step_fail(e)` used by the retry-with-halving loop.

The dimension-specific subclasses `Grid1D`/`Grid2D`/`Grid3D` each own one `FluidArray1D/2D/3D w` (Section 1.2.2) as their primary state, plus a staggered `MagneticArray2D/3D B` in MHD builds (Section 2.9). The magnetic field is stored on a staggered face-centred grid, with `B[i,j,k].x` corresponding to $B^x_{i-\frac{1}{2},j,k}$, `B[i,j,k].y` to $B^y_{i,j-\frac{1}{2},k}$, and `B[i,j,k].z` to $B^z_{i,j,k-\frac{1}{2}}$. External functions can access a reference to the staggered magnetic array via `grid._B()`.

1.3 Dependencies

DRAGON's only dependency is **HDF5** (The HDF Group, 1997–2026), used by DRAGONHOARD (Chapter 5)[4] for output and restart I/O. No other third-party library is required to build or run DRAGON itself.

DRAGON's in-house visualisation toolkit DRAGONGAZE (Chapter 7) depends separately on **h5py** (to read the HDF5 files DRAGONHOARD writes) and **Matplotlib** (to render plots from that data). These are Python-side dependencies only and do not affect the C++ build.

On macOS, HDF5 can be installed with Homebrew:

```
brew install hdf5
brew --prefix hdf5
```

[2] It is for this reason that DRAGON requires C++23

[3] Or, in the case of domain decomposition (Section 4.3), it might be a cell owned by another subgrid.

[4] Core DRAGON and DRAGONWING are agnostic to the file storage format, so if HDF5 is for some reason unavailable on your system, you would only need to modify DRAGONHOARD to interface with your alternative format.

The second command prints the path to the HDF5 installation on your computer. This will usually `/opt/homebrew/opt/hdf5` on Apple Silicon Macs, or `/usr/local/opt/hdf5` on Intel Macs.

On Linux, install HDF5 through your system package manager – for example, on Debian/Ubuntu:

```
sudo apt install libhdf5-dev
```

1.4 Style Guide

This section contains a style guide for developers who wish to contribute to DRAGON.

1.4.1 File Headers

Every file opens with a comment block:

```
//
//  <Filename>
//  <DirectoryPath>
//
//  Created by <original developer> on <creation date>.
//  <citations, if applicable>
//
```

Here `<Filename>` is the name of the file (including extension), and `<DirectoryPath>` is relative to the project root.

Each in-code citation should include author names, year, and a link. arXiv links are preferred if available, as this makes the papers available to users who don't belong to an institution. For sources not on arXiv, use a doi link if possible. If you implemented something based on the description in source A's description of a method first published in source B, you should cite as follows:

```
//  Implemented based in part on <citation for source A>
//      <citation(s) for source(s) B>
```

In addition to comment citations, also give a full citation in the bibliography of this document.

1.4.2 Header guards

The traditional pattern is preferred. Typically, the you should name it after the file stem.

```
#ifndef Grid_hpp
#define Grid_hpp
...
#endif
```

1.4.3 Includes

Includes should be grouped in the following order, leaving a blank line between each group

1. Header files defining things that this file implements
2. Includes which are used in a major way.
3. Includes which are used in a minor way.
4. Includes which are only used in error paths.

For header files other than in the first group, you should also include a trailing comment saying *why* this header is included. Trailing comments in the same group should have their starting points vertically aligned. Comments may be omitted if the file is being used for a data type which shares its name with the include, and for the module's configuration header.

For example, the includes in `CFL.cpp` are

```
#include "CFL.hpp"

#include "Config.h"
#include <cmath>        //For std::abs, sqrt, pow
#include <algorithm>    //For std::min/max

#include "Boundary/GhostFill.hpp"   //For boundary.apply

#include <stdexcept>    //For std::runtime_error (CFL timestep fails to compute)
#include <exception>    //For handling step restarts
#include <iostream>     //For printing step-restart messages
#include <cstdlib>      //For exit() if the timestep gets too small
```

This file implements everything declared in `CFL.hpp`. `Config.h` defines which of this file's calculation methods should be active, and the various `std::` functions are used heavily throughout the implmentation, so these includes qualify as major usage. `boundary.apply` meanwhile is only called in a couple of places, so it is considered a minor item. Lastly, the last four includes are only used to handle step restarts, so they are considered error-path-only.

Immediately after the includes, source files typically contain `using namespace DRAGON;` (and/or any other namespace they live in). Each of the top-level modules has its own namespace of the same name: `DRAGON`, `DRAGONWING`, `DRAGONHOARD`. Sub-areas get nested namespaces rather than longer names: `DRAGON::CONFIG`, `DRAGON::Boundary`, etc.

1.4.4 Naming

Types are PascalCase: `PrimitiveSate`, `Grid1D`, `ArrayGuard`. As an exception, `vec3` is lowercase. Namespaces are also PascalCase, except for top-level modules since those are acronyms.

Functions are typically either camelCase (e.g. `swapXY`, `getSize`, `waitForCompletion`) or snake_case (e.g. `advance_split`, `cfl_time`, `verify_and_fallback`). Named algorithms should however defer to the capitalisation seen in the literature (e.g. `HLL`, `Roe`, `MUSCL`, `CTU`).

Local variables should stay short, and where possible should match symbols used in the physics (e.g. `rho`, `B`, `dx`, `nx`). However, variable names should avoid conflicting with other variable names if this could lead to confusion (e.g. momentum is `mom`, since `p` already refers to pressure). Loop indices should generally be `i`, `j`, `k` for x, y, and z dimensions, plus `g` for ghost indices.

Compile-time options in `Config.h` (and corresponding files in other modules) should be SCREAMING_SNAKE_CASE (e.g. `RIEMANN_HLLX`, `CT_CONSV_TOTAL_E`) for preprocessor options and snake_case (e.g. `timestep_tolerance`, `ct_energy_beta`).

Derived Constants in `Constants.h` use the format `_A_B` to refer to the fraction A/B. For example, `_1_4pi`= $\frac{1}{4\pi}$, or `_Gm1_2G`= $\frac{\gamma-1}{2\gamma}$. Nonfractional constants may also include an underscore to avoid naming conflicts, such as `_gamma`.

1.4.5 Indentation

Indent using four-space tabs[5]. This is partly because I am used to Xcode's default, and partly because the "You're nesting too much" crowd isn't usually nesting three dimensions of loops.

`Config.h` uses a specialindentation to encode which options belong to which setting:

- A `#define` indented and sitting *above* another `#define` is one of the enumerated choices for that setting (e.g. `RIEMANN_HLLC` is a choice for `RIEMANN_DEFAULT`).
- A `#define` or `const` indented and sitting *below* another `#define` is a parameter specific to that setting (e.g. `exact_riemann_max_iters` only matters when `RIEMANN_EXACT` is selected).

This is spelled out in further detail in a comment at the top of the file.

Outside of `Config.h`, one should generally follow standard C++ indentation practices, with the following additional prescriptions:

1. Lines which purely serve an auxiliary purpose (e.g. calling `DRAGONWING` to allocate or release scratch memory) should be indented one extra line to keep the primary algorithm easier to read.
2. `#if`/`#ifdef` blocks should be at the same indentation level as both the surrounding and enclosed code. As such nested `#if`'s should generally be on the same indentation level.
3. `case` statements should be at the *same* indentation level as the `switch`. If a particular case requires multiple lines, indent after the first.

1.4.6 Braces

Opening braces should stay on the same line as the function signature or control statement. Closing braces should be on their own line, except that `else` or `while` should be a single space come after the closing brace of an `if` or `do` statement (respectively, and when applicable).

For `if-else` statements, either all should use braces or none should. Except to compliance with the preceding guideline, `if` statements with a single statement do not need braces, even if the statement is placed on its own line.

1.4.7 Spaces

Spaces should be included after the following keywords

```
1 if else switch case do while for
```

With most of these keywords you should also include a space before the opening brace. For functions however, there should be no space between the closing parentheses and the opening brace, unless there is some other keyword (e.g. `const`) between them, in which case that keyword should have a space on both sides.

Generally you should use spaces on each side any of the following operators:

```
1 = + - < > * / % | & ^ <= >= == != ? :
```

However, sometimes omitting some of these spaces can make larger mathematical statements easier to read by conveying order of operations. If this is the case, do that. For loops are another place where omitting these extra spaces is generally acceptable.

[5]Here in the Cult of the DRAGON, we are heretics.

Chapter 2

Physical Model & Numerical Methods

2.1 Governing Equations

The ideal MHD equations solved by DRAGON are, in conservative form:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{v}) = 0 \tag{2.1a}$$

$$\frac{\partial}{\partial t}(\rho \mathbf{v}) + \nabla \cdot \left[\rho \mathbf{v}\mathbf{v} - \frac{1}{4\pi}\mathbf{B}\mathbf{B} + \left(p + \frac{\mathbf{B}^2}{8\pi}\right)\mathbb{I}\right] = 0 \tag{2.1b}$$

$$\frac{\partial E}{\partial t} + \nabla \cdot \left[\left(E + p + \frac{\mathbf{B}^2}{8\pi}\right)\mathbf{v} - (\mathbf{v}\cdot\mathbf{B})\mathbf{B}\right] = 0 \tag{2.1c}$$

$$\frac{\partial \mathbf{B}}{\partial t} = \nabla \times (\mathbf{v} \times \mathbf{B}) \tag{2.1d}$$

subject to the constraint

$$\nabla \cdot \mathbf{B} = 0. \tag{2.1e}$$

Here ρ is mass density, $\mathbf{v}$ the fluid velocity, $\mathbf{B}$ the magnetic field, and p the thermal pressure. E the total energy density

$$E = \frac{p}{\gamma - 1} + \frac{1}{2}\rho \mathbf{v}^2 + \frac{\mathbf{B}^2}{8\pi}, \tag{2.2}$$

where γ adiabatic index. DRAGON adopts Gaussian (non-rationalized) electromagnetic units, so the magnetic pressure and Lorentz force terms carry an explicit factor of 4π.

2.2 Initial Conditions

In DRAGON, problem initialisation consists of three steps:

1. Hydrodynamic Initialisation
2. Magnetic Field Initialization
3. (Optional) Custom Additional Initialisation

In the first step, the user implements `Problem::initialFluidState(x,y,z)` which returns a primitive hydrodynamic state $w = (\rho, \mathbf{v}, p)$ at position (x, y, z). The cell $w_{i,j,k}$ will be initialised with the state computed by this function with input position $((i + \frac{1}{2})\,\mathrm{d}x, (j + \frac{1}{2})\,\mathrm{d}y, (k + \frac{1}{2})\,\mathrm{d}z)$. Any value of $\mathbf{B}$ provided in this step will be overridden in the following step, though there are cases where it is useful to compute a preliminary value of $\mathbf{B}$ at this stage.

In the second step, the user implements `Problem::initialMagneticPotential(x,y,z)` which returns vector potential $\mathbf{A}$ at position (x, y, z). From this function, $\mathbf{B} = \nabla \times \mathbf{A}$ is initialised as follows:

1. $\mathbf{A}$ is calculated at every edge on the grid.

$$A^x_{i,j-\frac{1}{2},k-\frac{1}{2}} = \mathbf{A}\left[\left(i + \frac{1}{2}\right)\mathrm{d}x, j\,\mathrm{d}y, k\,\mathrm{d}z\right]_x \tag{2.3a}$$

$$A^y_{i-\frac{1}{2},j,k-\frac{1}{2}} = \mathbf{A}\left[i\,\mathrm{d}x, \left(j + \frac{1}{2}\right)\mathrm{d}y, k\,\mathrm{d}z\right]_y \tag{2.3b}$$

$$A^z_{i-\frac{1}{2},j-\frac{1}{2},k} = \mathbf{A}\left[i\,\mathrm{d}x, j\,\mathrm{d}y, \left(k + \frac{1}{2}\right)\mathrm{d}z\right]_z \tag{2.3c}$$

2. The normal component of $\mathbf{B}$ is calculated on each of the faces

$$B^x_{i-\frac{1}{2},j,k} = \frac{1}{4}\left(A^y_{i-\frac{1}{2},j,k-\frac{1}{2}} + A^z_{i-\frac{1}{2},j+\frac{1}{2},k} - A^y_{i-\frac{1}{2},j,k+\frac{1}{2}} - A^z_{i-\frac{1}{2},j-\frac{1}{2},k}\right) \tag{2.4a}$$

$$B^y_{i,j-\frac{1}{2},k} = \frac{1}{4}\left(A^z_{i-\frac{1}{2},j-\frac{1}{2},k} + A^x_{i,j-\frac{1}{2},k+\frac{1}{2}} - A^z_{i+\frac{1}{2},j-\frac{1}{2},k} - A^x_{i,j-\frac{1}{2},k-\frac{1}{2}}\right) \tag{2.4b}$$

$$B^z_{i,j,k-\frac{1}{2}} = \frac{1}{4}\left(A^x_{i,j-\frac{1}{2},k-\frac{1}{2}} + A^y_{i+\frac{1}{2},j,k-\frac{1}{2}} - A^x_{i,j+\frac{1}{2},k-\frac{1}{2}} - A^y_{i-\frac{1}{2},j,k-\frac{1}{2}}\right) \tag{2.4c}$$

3. $\mathbf{B}_{i,j,k}$ is calculated as an average of the adjacent faces

$$B^x_{i,j,k} = \frac{1}{2}\left(B^x_{i-\frac{1}{2},j,k} + B^x_{i+\frac{1}{2},j,k}\right) \tag{2.5a}$$

$$B^y_{i,j,k} = \frac{1}{2}\left(B^y_{i,j-\frac{1}{2},k} + B^y_{i,j+\frac{1}{2},k}\right) \tag{2.5b}$$

$$B^z_{i,j,k} = \frac{1}{2}\left(B^z_{i,j,k-\frac{1}{2}} + B^z_{i,j,k+\frac{1}{2}}\right) \tag{2.5c}$$

This procedure ensures that $\mathbf{B}$ is initialised with zero divergence everywhere.

In the final step, the user implements `Problem::completeProblemInit(grid)` to perform any additional initialisation steps on `grid`.

2.3 Boundary Conditions

DRAGON supports several types of boundary conditions, and provides capability for defining customisable boundary conditions. All boundary conditions are subclasses of the `GhostFill` subclass, which are responsible for filling in the ghost cells on a `Grid` object. To set the boundary conditions for a problem, set the `boundary` property of the `Grid` class, for example

```
grid.boundary = Periodic();
```

Every time the grid advances, it will call the `apply(Grid&)` function on its boundary object.

A boundary object can be restricted to specific faces by passing a string argument to the constructor. In this string, specify any combination of X, Y, Z, optionally adding plus or minus after any or all of these to only include the respective side[1]. For example, `Reflective("XZ-")` uses reflective boundary conditions on the X_-, X_+, and Z_- boundaries. Any faces not explicitly specified will default to outflow.

Boundary conditions can be combined using the `+` operator. For example:

```
grid.boundary = Reflective("XZ-") + Periodic("Y");
```

defines a box which is reflective in the $X_\pm$ and Z_- boundaries, periodic in the Y direction, and defaults to outflow on the Z_+ boundary. Boundaries are applied in the order they are specified, with any implicit outflow being applied first. In the example, the reflective boundaries are applied before the periodic boundaries.

In addition to passing a face-restriction string, the GhostFill constructor also takes a boolean `corner_ghosts`. This value is true by default, and you should normally leave it that way. If `corner_ghosts` is true, then all ghosts on the corresponding side of the boundary will be filled. If it is false, then only the ghosts which are out of bounds in a single dimension are filled, and ghosts in the corner regions are assumed to have already been properly filled by a previous boundary object.

DRAGON includes the following built in boundary classes

2.3.1 Outflow

The `Boundary::Outflow` class implements outflow (Neumann) boundary conditions, filing each ghost with the state of the nearest physical cell. Outflow boundaries are the default boundary if none is set. Negative-side and positive-side ghosts on an X boundary are filled as

$$\begin{pmatrix} \rho_{(-g,j,k)} \\ \mathbf{v}_{(-g,j,k)} \\ p_{(-g,j,k)} \\ B^y_{(-g,j,k)} \\ B^z_{(-g,j,k)} \end{pmatrix} = \begin{pmatrix} \rho_{(0,j,k)} \\ \mathbf{v}_{(0,j,k)} \\ p_{(0,j,k)} \\ B^y_{(g-1,j,k)} \\ B^z_{(g-1,j,k)} \end{pmatrix}, \qquad \begin{pmatrix} \rho_{(n_x-1+g,j,k)} \\ \mathbf{v}_{(n_x-1+g,j,k)} \\ p_{(n_x-1+g,j,k)} \\ B^y_{(n_x-1+g,j,k)} \\ B^z_{(n_x-1+g,j,k)} \end{pmatrix} = \begin{pmatrix} \rho_{(n_x-1,j,k)} \\ \mathbf{v}_{(n_x-1,j,k)} \\ p_{(n_x-1,j,k)} \\ B^y_{(n_x-1,j,k)} \\ B^z_{(n_x-1,j,k)} \end{pmatrix} \tag{2.6}$$

Normal magnetic fields however are linearly extrapolated to preserve $\nabla \cdot \mathbf{B} = 0$:

$$B^x_{(-g,j,k)} = 2B^x_{(-g+1,j,k)} - B^x_{(-g+2,j,k)} \tag{2.7a}$$

$$B^x_{(n_x+g,j,k)} = 2B^x_{(n_x-1+g,j,k)} - B^x_{(n_x-2+g,j,k)} \tag{2.7b}$$

Calling `Boundary::Outflow::Gated` instead of the outflow constructor will return a modified outflow condition designed to inhibit accidental inflow. This variant follows all outflow criteria as normal, except that if the normal velocity is directed inwards to the physical region ($v_x > 0$ on an X_- boundary, $v_x < 0$ on an X_+ boundary), that component will be set to zero in the boundary region.

2.3.2 Periodic

The `Boundary::Periodic` class implements periodic boundary conditions, filling ghosts such that the ghost cells near the positive boundary match the physical cells near the corresponding negative boundary. Negative-side ghosts are filled as $w_{-g,j,k} = w_{n_x-g,j,k}$ and positive-side ghosts are filled as $w_{n_x+g,j,k} = w_{g,j,k}$. Face-magnetic ghosts are filled likewise for half-integer values of g.

[1] Plus and minus restriction is not supported for periodic boundary conditions

Note that periodic boundary conditions require that both the plus and minus boundaries of any given dimension are used; if restriction such as `Periodic("X-")` is attempted, the object will initialise such that sides will be used (in this case, equivalent to `Periodic("X")`).

2.3.3 Reflective

The `Boundary::Reflective` class implements reflective boundary conditions, filling ghosts such that they mirror the nearby physical cells. These conditions mimic a stationary wall, which in MHD is also taken to be perfectly conductive. Negative-side and positive-side ghosts on an X boundary are filled as

$$\begin{pmatrix} \rho_{(-g,j,k)} \\ v^x_{(-g,j,k)} \\ v^y_{(-g,j,k)} \\ v^z_{(-g,j,k)} \\ p_{(-g,j,k)} \\ B^x_{(-g,j,k)} \\ B^y_{(-g,j,k)} \\ B^z_{(-g,j,k)} \end{pmatrix} = \begin{pmatrix} \rho_{(g-1,j,k)} \\ -v^x_{(g-1,j,k)} \\ v^y_{(g-1,j,k)} \\ v^z_{(g-1,j,k)} \\ p_{(g-1,j,k)} \\ B^x_{(g-1,j,k)} \\ -B^y_{(g-1,j,k)} \\ -B^z_{(g-1,j,k)} \end{pmatrix}, \qquad \begin{pmatrix} \rho_{(n_x-1+g,j,k)} \\ v^x_{(n_x-1+g,j,k)} \\ v^y_{(n_x-1+g,j,k)} \\ v^z_{(n_x-1+g,j,k)} \\ p_{(n_x-1+g,j,k)} \\ B^x_{(n_x-1+g,j,k)} \\ B^y_{(n_x-1+g,j,k)} \\ B^z_{(n_x-1+g,j,k)} \end{pmatrix} = \begin{pmatrix} \rho_{(n_x-g,j,k)} \\ -v^x_{(n_x-g,j,k)} \\ v^y_{(n_x-g,j,k)} \\ v^z_{(n_x-g,j,k)} \\ p_{(n_x-g,j,k)} \\ B^x_{(n_x-g,j,k)} \\ -B^y_{(n_x-g,j,k)} \\ -B^z_{(n_x-g,j,k)} \end{pmatrix} \tag{2.8}$$

Face-magnetic ghosts are filled likewise for half-integer values of g.

2.3.4 Fixed

The `Boundary::Fixed` class implements fixed (Dirichlet) boundary conditions, filling ghosts with a constant uniform state. When constructing a fixed boundary, the primitive fluid state is the first argument.

Fixed boundaries are fine to use in hydrodynamic problems, but aren't recommended for non-trivial MHD problems since their interaction with constrained transport hasn't been tested yet.

2.3.5 Ignore

The `Boundary::Ignore` class is a no-op boundary. Generally you should not use this in your problems, but the it is used internally to support domain decomposition.

2.4 Multidimensional Integration Schemes

2.4.1 Operator-Split (Strang) Update

In split mode, each timestep is built from a sequence of 1D sweeps along coordinate directions following the dimensional-splitting approach of Strang (1968), each solved with the same 1D Godunov kernel (`Godunov::sweep`): apply boundary conditions (Section 2.3), reconstruct interface states with MUSCL (Section 2.6), solve the Riemann problem (Section 2.7) at each face, and difference the resulting fluxes into every cell in a single pass down the array, reusing the right-going flux of one interface as the left-going flux of the next.

A multidmensional split sweep consists of sweeps in the x, y, and (if applicable) z directions. A sweep along y or z is performed by transposing (via `swappedXY` or `swappedXZ`) each 1D line of cells so the swept direction is treated as x, running the same kernel as the x sweep, and transposing back. This approach means that only one 1D solver needs to be implemented and tested. Each split

step operates on a scratch copy of the grid so that, if any sub-sweep fails the physicality check, the original state is left untouched and the whole step can be retried at a smaller Δt.

To avoid the directional bias a fixed sweep order would introduce, DRAGON alternates or rotates the sweep order with every step. In two dimensions, this is done by alternating by between XYX and YXY, both with a $(\frac{\Delta t}{2}, \Delta t, \frac{\Delta t}{2})$ pattern. In three dimensions, this is done by rotating through six orderings: the cyclic permutations $(XYZYX)$, $(ZXYXZ)$, $(YZXZY)$, and the anticyclic permutations $(ZYXYZ)$, $(XZYZX)$, and $(YXZXY)$ all with a $(\frac{\Delta t}{2}, \frac{\Delta t}{2}, \Delta t, \frac{\Delta t}{2}, \frac{\Delta t}{2})$ pattern.

2.4.2 Unsplit (CTU) Update

The unsplit driver (`Grid::unsplit_step`) advances all directions simultaneously in a single conservative update, following this sequence:

1. **MUSCL** (Section 2.6): compute preliminary half-step interface states in every direction.
2. **Preliminary Fluxes** (Section 2.7): Solve the Riemann problem at every face using the preliminary interface states.
3. **Preliminary Sources** (Chapter 3): Compute the non-stiff source terms $S(w_i^n)$.
4. **Half-Step Predictor**: Perform steps 7-8 using the preliminary fluxes and sources, with the timestep halved and also including one ghost layer. Store the result to a scratch copy $w^{n+\frac{1}{2}}$.
5. **CTU** (Section 2.6.4): apply transverse corrections to the interface states from step 1, then recompute the fluxes from step 2. Skip this step if `CTU` is disabled in `Config.h`
6. **Corrected Sources** (Chapter 3): Recompute the non-stiff source terms at each physical cell, using $w_i^{n+\frac{1}{2}}$ instead of w_i^n as the inputs.
7. **Update Hydrodynamic States**: For each physical cell, apply the final fluxes sources conservatively, storing the result in a scratch copy.
8. **Constrained Transport** (Section 2.9): extract, upwind, and integrate the electric field, then advance the face-normal $\mathbf{B}$ and recompute body-centred fields.
9. **Physicality check**: if any updated cell is not physical, throw to trigger a step restart.
10. **Passive Scalars** (Section 2.10): Use the final mass fluxes to advect passive scalars.
11. **Synchronization**: in a domain-decomposed run, wait for all other subgrids to reach the same checkpoint (and confirm none of them failed) before proceeding.
12. **Commit**: copy the scratch states computed by steps 7, 8, and 10 back into the live grid.

For the flux application in step 8, `Godunov::applyFluxes` takes fluxes F^X, F^Y, F^Z through every face of a cell, plus the body source S, and applies them as

$$U_{i,j,k}^{n+1} = U_{i,j,k}^n + \left((\nabla \cdot \mathbf{F})_{i,j,k} + S_{i,j,k}\right) \Delta t \tag{2.9a}$$

$$(\nabla \cdot \mathbf{F})_{i,j,k} = \frac{1}{\Delta x}\left(F^X_{i-\frac{1}{2},j,k} - F^X_{i+\frac{1}{2},j,k}\right) + \frac{1}{\Delta y}\left(F^Y_{i,j-\frac{1}{2},k} - F^Y_{i,j+\frac{1}{2},k}\right) + \frac{1}{\Delta z}\left(F^Z_{i,j,k-\frac{1}{2}} - F^Z_{i,j,k+\frac{1}{2}}\right) \tag{2.9b}$$

The sum $(\nabla \cdot \mathbf{F})_{i,j,k} + S_{i,j,k}$ is computed as a conservative change, which is then applied via the addition procedure described in Section 2.8. The update throws if any updated cell is non-finite (this is a coarser, cheaper check than the full physicality test applied after the CT update).

2.5 Time Integration

2.5.1 Per-Cell Signal Speed

For a single cell with primitive state w, DRAGON computes a local CFL signal speed from the fast magnetosonic speed c_f (sound speed c_s in hydrodynamic builds), combined across the active grid directions in one of three ways, selected by `CFL_CALCULATION` in `Config.h`:

$$\texttt{CFL_MAX:} \qquad s_{\max} = \max_{d\in\{x,y,z\}} \left(\frac{|v_d| + c}{\Delta d} \right), \tag{2.10}$$

$$\texttt{CFL_ADD:} \qquad s_{\text{add}} = \sum_{d\in\{x,y,z\}} \frac{|v_d| + c}{\Delta d}, \tag{2.11}$$

$$\text{power } p: \qquad s_p = \left[\sum_{d\in\{x,y,z\}} \left(\frac{|v_d| + c}{\Delta d} \right)^p \right]^{1/p}, \tag{2.12}$$

where a direction d is only included if $\Delta d > 0$ (i.e. it is an active grid dimension). Equation 2.10 is typically appropriate for dimensional splitting, while the more conservative equation 2.11 is appropriate for unsplit updates where waves can cross diagonally in a single step. Equation 2.12 interpolates between the two as p is tuned, approaching equation Equation (2.10) as $p \to \infty$.

2.5.2 Grid-Wide Timestep

The global timestep bound is derived from the inverse of the fastest signal speed for any one cell in the physical region of the grid:

$$\Delta t \le C_{\text{CFL}} \min_{i,j,k} \left(\frac{1}{s_{i,j,k}} \right) \tag{2.13}$$

where $s_{i,j,k}$ are the speeds calculated in the previous section for each individual cell and $C_{\text{CFL}} \le 1$ is the coefficient given by `CONFIG::CFL_coeff`. The first layer of ghosts are also included in the calculation, but additional ghosts are ignored as they are only used to correct the states seen at the interfaces. If the resulting timestep is nonpositive, `CFL::cfl_time` throws an exception, since no finite timestep can be justified in that case.

The `Grid::advance(dt)` function is responsible for controlling the time advancement as follows

1. Each step begins with the application of boundary conditions.
2. Timestep Δt_1 is taken to be the lesser of `dt` and the bound given by 2.13.
3. Advance the problem by time Δt_1. If an exception is thrown, halve Δt_1 and try again.
4. Subtract Δt_1 from `dt`.
5. Repeat from step 1 until `dt` falls below `CONFIG::Timestep_Tolerance` (10^{-14} by default)

This retry-with-halving behaviour is what lets the physicality checks recover from an occasional bad step (e.g. from an overly aggressive CFL coefficient) without aborting the whole simulation, at the cost of re-doing the failed substep at a smaller Δt.

2.6 Reconstruction

DRAGON reconstructs left/right interface states using the MUSCL (Monotonic Upstream-centered Scheme for Conservation Laws) approach of van Leer (1979), applied to the primitive variables $w = (\rho, \mathbf{v}, p, \mathbf{B})$.

2.6.1 Piecewise-Linear Slopes

For each cell i, a limited slope Δ_i is computed from the neighbouring cell averages,

$$\Delta_i = \text{limiter}(w_i - w_{i-1},\ w_{i+1} - w_i)\,, \tag{2.14}$$

where w_i are the primitive variables in the ith cell, and limiter is the limiter function selected by setting `MUSCL_DEFAULT_LIMITER` in `Config.h`. The following limiter functions are implemented (Roe, 1986; van Leer, 1974, 1977; van Albada et al., 1982):

$$\text{minmod}(a,b) = \begin{cases} a & |a| < |b| \text{ and } ab > 0, \\ b & |b| \le |a| \text{ and } ab > 0, \\ 0 & ab \le 0. \end{cases} \tag{2.15a}$$

$$\text{MC}(a,b) = \begin{cases} 2a & |a| \le \left\{|b|, \tfrac{1}{4}|a+b|\right\} \text{ and } ab > 0, \\ 2b & |b| \le \left\{|a|, \tfrac{1}{4}|a+b|\right\} \text{ and } ab > 0, \\ \tfrac{1}{2}|a+b| & \tfrac{1}{4}|a+b| < \{|a|, |b|\} \text{ and } ab > 0, \\ 0 & ab \le 0. \end{cases} \tag{2.15b}$$

$$\text{superbee}(a,b) = \begin{cases} 2a & 2|a| \le |b| \text{ and } ab > 0, \\ 2b & 2|b| \le |a| \text{ and } ab > 0, \\ a & |b| < |a| < 2|b| \text{ and } |a| > |b| \text{ and } ab > 0, \\ b & |a| \le |b| < 2|a| \text{ and } |b| > |a| \text{ and } ab > 0, \\ 0 & ab \le 0. \end{cases} \tag{2.15c}$$

$$\text{vanLeer}(a,b) = \begin{cases} \frac{2ab}{a+b} & ab > 0, \\ 0 & ab \le 0. \end{cases} \tag{2.15d}$$

$$\text{vanAlbada}(a,b) = \begin{cases} \frac{ab(a+b)}{a^2+b^2} & ab > 0, \\ 0 & ab \le 0. \end{cases} \tag{2.15e}$$

The monotonized-central (MC) limiter (van Leer, 1977) is selected by default, as it provides a reasonable balance between robustness and nondiffusiveness.

Once Δ_i is computed, the piecewise-linear interface states are computed as

$$w^L_{i+1/2} = w_i + \tfrac{1}{2}\Delta_i, \qquad w^R_{i+1/2} = w_{i+1} - \tfrac{1}{2}\Delta_{i+1}. \tag{2.16}$$

Note that in code $w^L_{i+1/2}$ is generally referred to as `_xR[i]` and $w^R_{i+1/2}$ as `_xL[i+1]`, denoting them relative to the cells from which they originate rather than the Riemann problem to which they will be passed.

2.6.2 MUSCL-Hancock Time Predictor

To achieve second-order accuracy in time, the interface states are then advanced by a half timestep using the flux difference across the cell,

$$w_{i+1/2}^{L,n+1/2} = w_{i+1/2}^{L} + \frac{\Delta t}{2\Delta x}\left[F(w_{i-1/2}^{R}) - F(w_{i+1/2}^{L})\right] + \frac{\Delta t}{2}\left[S(w_i) + S_{\text{MHD}}\right], \tag{2.17a}$$

$$w_{i-1/2}^{R,n+1/2} = w_{i-1/2}^{R} + \frac{\Delta t}{2\Delta x}\left[F(w_{i-1/2}^{R}) - F(w_{i+1/2}^{L})\right] + \frac{\Delta t}{2}\left[S(w_i) + S_{\text{MHD}}\right] \tag{2.17b}$$

where $S(w_i)$ are user-defined source terms (non-stiff only), S_{MHD} are MHD source terms (see Section 2.6.3). Note that $w_{i+1/2}^{LR}$ are converted from primitive to conservative variables before the above operation, and then converted back to primitive variables when assigned to $w_{i+1/2}^{LR,n+1/2}$. DRAGON's fluid arithmetic allows this to be done implicitly, though MUSCL does the primitive-to-conservative conversion explicitly to avoid redundant conversion.

If for a given cell the above procedure produces a nonphysical result for $w_{i-1/2}^{R}$ or $w_{i+1/2}^{L}$, then both of these interface states fall back to the first-order reconstruction $w_{i-1/2}^{R} = w_{i+1/2}^{L} = w_i$. The above procedure can also be disabled (in which case first-order reconstruction is always used) by disabling `MUSCL_Hancock` in `Config.h`, though this isn't recommended.

2.6.3 MHD Source terms

When reconstructing MHD states, additional source terms S_{MHD} are needed to ensure the reconstructed state remains consistent with the solenoid constraint. These terms are computed from the face-normal fields $B_{i+1/2,j,k}^{x}$ (see section 2.9) and their normal derivatives

$$\frac{\partial B_x}{\partial x} = \frac{1}{\Delta x}\left(B_{i+1/2,j,k}^{x} - B_{i-1/2,j,k}^{x}\right) \tag{2.18a}$$

$$\frac{\partial B_y}{\partial x} = \frac{1}{\Delta y}\left(B_{i,j+1/2,k}^{y} - B_{i,j-1/2,k}^{y}\right) \tag{2.18b}$$

$$\frac{\partial B_z}{\partial x} = \frac{1}{\Delta z}\left(B_{i,j,k+1/2}^{z} - B_{i,j,k-1/2}^{z}\right) \tag{2.18c}$$

For the transverse magnetic field, the source terms are given as (Gardiner and Stone, 2008):[2].

$$S_{\text{MHD}}(B_y) = v_y\left[\text{minmod}\left(\frac{\partial B_x}{\partial x}, -\frac{\partial B_y}{\partial y}\right) - \frac{\Delta_i(B_x)}{\Delta x}\right] \tag{2.19}$$

and similarly for B_z. For the normal term however $S_{\text{MHD}}(B_x) = 0$, as $(B_x)_{i+1/2}^{L,n+1/2}$ and $(B_x)_{i+1/2}^{R,n+1/2}$ will be set to $B_{i+1/2}^{x}$ after MUSCL reconstruction is complete. The energy source term is then given by

$$S_{\text{MHD}}(E) = \frac{\mathbf{B}}{4\pi} \cdot S_{\text{MHD}}(\mathbf{B}) \tag{2.20}$$

where $\mathbf{B}$ are taken from w_i. MHD Source terms are not needed for density and momentum, as these terms do not interact with magnetic field outside of their fluxes.

[2]Note that minmod is used regardless of which limiter is used for equation 2.14.

2.6.4 CTU Transverse Correction

The plain MUSCL half-states of Section 2.6 account only for the wave structure normal to each interface; used directly in an unsplit update, they under-resolve genuinely multidimensional flow. DRAGON corrects for this following the procedure given by Gardiner and Stone (2008).

Hydrodynamics. The correction (`Godunov::correctState`) is a half-step transverse flux difference applied to *both* sides of each interface state,

$$w_d^{L,\mathrm{corr}} = w_d^L - \frac{\Delta t}{2\Delta d'}\left[F_{d'}(w_{d'+\frac{1}{2}}) - F_{d'}(w_{d'-\frac{1}{2}})\right], \qquad w_d^{R,\mathrm{corr}} = w_d^R - \frac{\Delta t}{2\Delta d'}\left[\cdots\right], \tag{2.21}$$

for each transverse direction $d' \neq d$. In 3D, Gardiner and Stone (2008)'s "12-solve" algorithm additionally corrects each transverse flux itself with a one-third-weighted contribution from the *third* direction before using it (`computeCTUFlux_X/Y/Z`), so that corner-crossing waves are not double-counted.

Magnetohydrodynamics. In MHD, DRAGON first computes a half-step-advanced $\mathbf{B}$ and body-centred $\mathbf{E}$ using preliminary, uncorrected fluxes (by solving the Riemann problem at the interfaces, see Section 2.7) and one CT update (see section 2.9) at $\Delta t/2$. From here, the conservative hydrodynamic variables $(\rho, \rho\mathbf{v}, E)$ are updated similarly as to the hydrodynamic case, except that corrections from both transverse dimensions are applied simultaneously instead of feeding one update into the other. Instead of fluxes, magnetic fields receive their corrections from the electric fields $\mathbf{E}$ computed in the preliminary CT update. For the x interface, this is given by:

$$B_{y,i\pm\frac{1}{2},j,k}^{LR,n+\frac{1}{2}} = B_{y,i\pm\frac{1}{2},j,k}^{LR,n} + \frac{\Delta t}{4\Delta z}\left[\mathbf{E}^x_{i,j-\frac{1}{2},k-\frac{1}{2}} - \mathbf{E}^x_{i,j-\frac{1}{2},k+\frac{1}{2}} + \mathbf{E}^x_{i,j+\frac{1}{2},k-\frac{1}{2}} - \mathbf{E}^x_{i,j+\frac{1}{2},k+\frac{1}{2}}\right] + SB_y \tag{2.22a}$$

$$B_{z,i\pm\frac{1}{2},j,k}^{LR,n+\frac{1}{2}} = B_{z,i\pm\frac{1}{2},j,k}^{LR,n} + \frac{\Delta t}{4\Delta y}\left[\mathbf{E}^x_{i,j+\frac{1}{2},k-\frac{1}{2}} - \mathbf{E}^x_{i,j-\frac{1}{2},k-\frac{1}{2}} + \mathbf{E}^x_{i,j+\frac{1}{2},k+\frac{1}{2}} - \mathbf{E}^x_{i,j-\frac{1}{2},k+\frac{1}{2}}\right] + SB_z \tag{2.22b}$$

and similarly for the y and z interfaces. The normal components of $\mathbf{B}$ meanwhile are set to equal to the face-normal values from the staggered grid.

However, the MHD correction additionally needs to keep the corrected interface states consistent with the CT electric field, following Gardiner and Stone (2008) eqs. 51–59. These additional[3] corrections are

$$SB_y = \frac{\Delta t}{2} v_y \mathrm{minmod}\left(\frac{\partial B_x}{\partial x}, -\frac{\partial B_z}{\partial z}\right), \qquad SB_z = \frac{\Delta t}{2} v_z \mathrm{minmod}\left(\frac{\partial B_x}{\partial x}, -\frac{\partial B_y}{\partial y}\right) \tag{2.23a}$$

$$SE = \frac{\Delta t}{8\pi}\left[B_y v_y \mathrm{minmod}\left(\frac{\partial B_x}{\partial x}, -\frac{\partial B_z}{\partial z}\right) + B_z v_z \mathrm{minmod}\left(\frac{\partial B_x}{\partial x}, -\frac{\partial B_y}{\partial y}\right)\right] \tag{2.23b}$$

for x-interfaces and similarly for y and z interfaces. The derivatives $\frac{\partial B_i}{\partial x_i}$ are the same as computed in section 2.6.3. Note that Gardiner and Stone (2008) also prescribe a momentum term, but our MUSCL implementation renders this term unnecessary.

[3] That is, these terms are in addition to, not in place of equations 2.22 and the corresponding hydrodynamic update

2.7 Riemann Solvers

The Riemann problem takes as inputs the left and right states w_L and w_R. In the context of a Godunov code, the relevant solution is the flux through the $x = 0$ plane. In DRAGON, this can be obtained by calling `Riemann(wL,wR).flux()`. This function uses whichever Riemann solver is selected by `RIEMANN_DEFAULT` in `Config.h`.

In multidimensional problems, the Riemann problem will also need to be solved on the y and z interfaces. `Riemann(wL,wR).flux_Y()` and `Riemann(wL,wR).flux_Z()` accomplish this by swapping x and y (or x and z), solving the problem as if it were an x interface, and then swapping the output back. `Riemann(wL,wR).flux_X()` is also provided for code symmetry, though this simply calls `Riemann(wL,wR).flux()` as no swapping is necessary.

2.7.1 Exact (Hydrodynamics Only)

The Exact Riemann solver provides an exact solution to the Hydrodynamic Riemann problem (Toro, 2009), but as it generally requires an iterative procedure it is more costly than the other solvers. This solver can be called directly as `Riemann(wL,wR).exact().flux()`[4]

The exact solver iterates on the velocity-jump function

$$f(p,w) = \begin{cases} (p-p_w)\sqrt{\dfrac{2/[(\gamma+1)\rho_w]}{p+\frac{\gamma-1}{\gamma+1}p_w}} & p > p_w \text{ (shock)}, \\ \dfrac{2c_{s,w}}{\gamma-1}\left[\left(\dfrac{p}{p_w}\right)^{\frac{\gamma-1}{2\gamma}} - 1\right] & p \le p_w \text{ (rarefaction)}, \end{cases} \tag{2.24}$$

to solve $f(p^*, w_L) + f(p^*, w_R) + (v_{x,R} - v_{x,L}) = 0$ for p^*.

When `Exact_Rarefactions_Check` is enabled in `Config.h`, DRAGON first tests whether the wave pattern will be two rarefactions ($f(p_{\min}, w_L) + f(p_{\min}, w_R) + v_{x,R} - v_{x,L} \ge 0$ with $p_{\min} = \min(p_L, p_R)$); if so, it uses the closed-form Two-Rarefaction Riemann Solver (TRRS) instead of iterating. Otherwise DRAGON starts from the initial guess $p_0^* = (p_L + p_R)/2$ and uses a damped Newton step ($\delta p^* = \min(f/f', 0.8\,p^*)$) to prevent divergence to negative pressure. The iteration terminates when the relative change in p^* falls below `CONFIG::ExactRiemann_Tolerance` or `CONFIG::ExactRiemann_MaxIters` is exhausted.

Once p^* is calculated, star-region velocity and densities are given by

$$v_x^* = \frac{1}{2}\left[v_{x,L} + v_{x,R} + f(p^*, w_R) - f(p^*, w_L)\right], \tag{2.25a}$$

$$\rho_K^* = \begin{cases} \rho_K \left(\dfrac{p^* + \frac{\gamma-1}{\gamma+1}p_K}{\frac{\gamma-1}{\gamma+1}p^* + p_K}\right) & p^* > p_K \text{ (shock)}, \\ \rho_K \left(p^*/p_K\right)^{1/\gamma} & p^* \le p_K \text{ (rarefaction)}, \end{cases} \qquad K \in \{L, R\}. \tag{2.25b}$$

From here, the solution is sampled along the requested ray x/t (outside the wave fan, inside a shock's jump, or inside a rarefaction fan) to produce the flux; the sampling routine handles the left side of the contact by mirroring the problem (swapping $L \leftrightarrow R$ and negating normal velocity) and solving as if it were the right side to ensure implementation symmetry.

[4] The intermediate object given by `Riemann(wL,wR).exact()` contains the star states as well as the initial problem; calling `flux(x_t)` on this object then computes the flux through the trajectory $x/t =$`x_t`, with `x_t=0` as the default.

2.7.2 HLL

The two-wave Harten–Lax–van Leer solver approximates the star region as a single uniform state bounded by a pair of waves with speeds S_L and S_R. This solver can be called as `Riemann(wL,wR).HLL()`, or if wave speed estimates are already known, `Riemann(wL,wR).HLL(SL,SR)`.

Normal Magnetic Fields

In MHD, it is important that both sides of the problem have the same normal component of B. For HLL and all other solvers that support MHD, this is done by setting both to $\frac{1}{2}\left(B_x^L + B_x^R\right)$.

Roe-Averaged Intermediate State

HLL and HLLC both use a shared Roe-averaged state M (`HLL::roeAvg`) to sharpen their outer wave-speed estimates:

$$\begin{pmatrix} \rho_M \\ \mathbf{v}_M \\ \mathbf{B}_M \\ H_M \\ p_M \end{pmatrix} = \begin{pmatrix} \sqrt{\rho_L \rho_R} \\ \dfrac{\sqrt{\rho_L}\,\mathbf{v}_L + \sqrt{\rho_R}\,\mathbf{v}_R}{\sqrt{\rho_L} + \sqrt{\rho_R}} \\ \dfrac{\sqrt{\rho_L}\,\mathbf{B}_L + \sqrt{\rho_R}\,\mathbf{B}_R}{\sqrt{\rho_L} + \sqrt{\rho_R}} \\ \dfrac{\sqrt{\rho_L}\,H_L + \sqrt{\rho_R}\,H_R}{\sqrt{\rho_L} + \sqrt{\rho_R}} \\ \left(1 - \frac{1}{\gamma}\right)\left(\rho_M H_M - \frac{1}{2}\rho_M \mathbf{v}_M^2\right) \end{pmatrix} \tag{2.26}$$

where $H_K = \frac{1}{\rho}\left(E_K + p_K + \frac{\mathbf{B}_K^2}{8\pi}\right)$ is specific enthalpy.

HLL Wave Estimates

Outer wave speeds combine the physical-state and Roe-averaged signal speeds,

$$S_L = \min\left(v_{x,L} - c_L,\ v_{x,M} - c_M\right), \qquad S_R = \max\left(v_{x,R} + c_R,\ v_{x,M} + c_M\right), \tag{2.27}$$

where c is the fast magnetosonic speed (sound speed in pure hydrodynamics) evaluated at L, R, and the Roe average M.

HLL Flux

Once the above speed estimates are calculated, the flux is then found via `Riemann(wL,wR).HLL(SL,SR)`[5]

The standard two-state flux is given by Harten et al. (1983):

$$F_{\mathrm{HLL}}(S_L, S_R) = \begin{cases} F_L & S_L \geq 0, \\ \frac{S_R F_L - S_L F_R + S_L S_R (U_R - U_L)}{S_R - S_L} & S_L < 0 < S_R, \\ F_R & S_R \leq 0, \end{cases} \tag{2.28}$$

where F_L and F_R are the fluxes computed directly from conservative states U_L and U_R respectively.

[5] If you have your own method for calculating the wave speed estimates, you can call this method yourself instead of using the existing HLL machinery.

2.7.3 HLLE

The HLLE solver implemented in `Riemann::HLLE()` uses the same two-state flux formula as HLL, but with the Einfeldt (1988) wave-speed bounds to additionally guarantee positivity of the intermediate density.

These speeds are given as

$$S_L = \min(v_{x,L} - c_L,\ v_{x,M} - d), \qquad S_R = \max(v_{x,R} + c_R,\ v_{x,M} + d), \tag{2.29}$$

where

$$d = \sqrt{\bar{a}^2 + \tfrac{1}{2}\sqrt{\rho_L \rho_R}\bar{v}_x^2}, \qquad \bar{a}^2 = \frac{\sqrt{\rho_L}c_L^2 + \sqrt{\rho_R}c_R^2}{\sqrt{\rho_L} + \sqrt{\rho_R}}, \qquad \bar{v}_x = \frac{v_{x,R} - v_{x,L}}{\sqrt{\rho_L} + \sqrt{\rho_R}}. \tag{2.30}$$

2.7.4 HLLC (Hydrodynamics Only)

HLLC (Toro et al., 1994) restores the contact discontinuity smeared out by HLL/HLLE.

`Riemann(wL,wR).HLLC()` uses the same outer wave speeds given in section 2.7.2, or `Riemann(wL,wR).HLLC(SL,SR)` can be called directly given alternative estimates. The speed of the restored contact wave is then given by

$$S_M = \frac{p_R^* - p_L^*}{\rho_R(v_{x,R} - S_R) - \rho_L(v_{x,L} - S_L)} \tag{2.31}$$

where $p_K^* = p_K + \rho_K v_{x,K}(v_{x,K} - S_K)$.

Using this speed, identify the relevant side $X \in \{L, R\}$[6] and outer speed S_X . From there, the star-region conservative state is

$$U^* = U_X \left(\frac{S_X - v_{x,X}}{S_X - S_M} \right), \tag{2.32}$$

except that $(\rho v_x)^* = \rho^* S_M$ and E^* is given by

$$E^* = E_K \left(\frac{S_X - v_{x,X}}{S_X - S_M} \right) + p_X \left(\frac{S_M - v_{x,X}}{S_X - S_M} \right) + \rho^*(S_M - v_{x,X})S_M \tag{2.33}$$

From here, the final star-state flux is given by

$$F = F_X + S_X(U^* - U_X). \tag{2.34}$$

2.7.5 HLLD (MHD Only)

HLLD (Miyoshi and Kusano, 2005) extends the contact-resolving idea of HLLC to MHD by resolving five waves: the two outer fast waves (S_L, S_R), two rotational/Alfvén waves (S_L^*, S_R^*), and the contact wave (S_M).

Wave speeds

Unlike other HLL solvers which use rho-averaging, HLLD's outer speeds use the fast magnetosonic speed c_f directly,

$$S_L = \min(v_{x,L} - c_{f,L},\ v_{x,R} - c_{f,R}), \qquad S_R = \max(v_{x,L} + c_{f,L},\ v_{x,R} + c_{f,R}). \tag{2.35}$$

If $S_L \geq 0$ or $S_R \leq 0$, then as with other HLL solvers the flux is simply F_L or F_R.

[6] $X = L$ if $S_M > 0$, otherwise $X = R$

The contact wave speed is given by

$$S_M = \frac{(\varrho_R v_{x,R} - \varrho_L v_{x,L}) - (p_{T,R} - p_{T,L})}{\varrho_R - \varrho_L} \tag{2.36}$$

where $p_{T,K} = p_K + \frac{1}{8\pi} B_K^2$ is the total pressure and $\varrho_K = \rho_K(S_K - v_{x,K})$ are the mass fluxes. Alfvén speeds and star densities are then

$$S_L^* = S_M - \frac{|B_x|}{\sqrt{4\pi\rho_L^*}}, \qquad S_R^* = S_M + \frac{|B_x|}{\sqrt{4\pi\rho_R^*}}, \qquad \rho_K^* = \frac{\varrho_K}{S_K - S_M}. \tag{2.37}$$

Outer Star States

Given total pressures $p_T^* = \frac{\varrho_R p_{T,L} - \varrho_L p_{T,R} + \varrho_L \varrho_R (v_{x,R} - v_{x,L})}{\varrho_R - \varrho_L}$. and $p_{T,K} = p_K + \frac{\mathbf{B}_K^2}{8\pi}$, the outer star states are given by

$$\rho_K^* = \frac{\varrho_K}{S_K - S_M} \tag{2.38a}$$

$$(v_x)_K^* = S_M \tag{2.38b}$$

$$(v_y)_K^* = v_y - \frac{B_{y,K}}{4\pi}\left[\frac{B_x(S_M - v_{x,K})}{\varrho_K(S_K - S_M) - \frac{B_x^2}{8\pi}}\right] \tag{2.38c}$$

$$(B_x)_K^* = B_x \tag{2.38d}$$

$$(B_y)_K^* = B_{y,K}\left[\frac{\varrho_K(S_K - v_{x,K}) - \frac{B_x^2}{8\pi}}{\varrho_K(S_K - S_M) - \frac{B_x^2}{8\pi}}\right] \tag{2.38e}$$

$$E_K^* = E_K\left(\frac{S_K - v_{x,K}}{S_K - S_M}\right) + \frac{p_T^* S_M - p_{T,K} v_{x,K}}{S_K - S_M} + \frac{Bx}{4\pi}\frac{(\mathbf{v}_K \cdot \mathbf{B}_K - \mathbf{v}_K^* \cdot \mathbf{B}_K^*)}{S_K - S_M} \tag{2.38f}$$

and similarly for the z components. If the denominators are zero (or sufficiently close to zero), then the transverse components of $\mathbf{v}$ and $\mathbf{B}$ are taken directly from w_K without rescaling.

Sometimes HLLD will compute a star state with nonpositive thermal pressure. If `HLLD_PHYSICAL_SAFETY` is enabled in `Config.h`, then DRAGON will compute the thermal pressure for the star states and switch to HLLE for a nonphysical result.

If the zero line falls between an alfven speed S_K^* and the corresponding outer speed, S_K, then the flux is given by

$$F_K^* = F_K + S_K(U_K^* - U_K) \tag{2.39}$$

on the appropriate side.

Inner Star States

The inner star states are

$$\rho_K^{**} = \rho_K^* \tag{2.40a}$$

$$(v_x)_K^{**} = S_M \tag{2.40b}$$

$$(v_y)_K^{**} = \frac{\sqrt{\rho_L^*} v_{y,L}^* + \sqrt{\rho_L^*} v_{y,R}^*}{\sqrt{\rho_L^*} + \sqrt{\rho_R^*}} \pm \frac{B_{y,R} - B_{y,L}}{\sqrt{4\pi\rho_L^*} + \sqrt{4\pi\rho_L^*}} \tag{2.40c}$$

$$(B_x)_K^{**} = B_x \tag{2.40d}$$

$$(B_y)^{**}_K = \frac{\sqrt{\rho^*_L}B^*_{y,R} + \sqrt{\rho^*_L}B^*_{y,L}}{\sqrt{\rho^*_L} + \sqrt{\rho^*_R}} \pm \frac{v_{y,R} - v_{y,L}}{\sqrt{\frac{1}{4\pi\rho^*_L}} + \sqrt{\frac{1}{4\pi\rho^*_R}}} \tag{2.40e}$$

$$E^{**}_K = \mp \left(\mathbf{v}^*_K \cdot \mathbf{B}^*_K - \mathbf{v}^{**}_K \cdot \mathbf{B}^{**}_K\right) \sqrt{\frac{\rho^*_K}{4\pi}} \tag{2.40f}$$

and similarly for the z components. For the transverse components, the + path is taken if $B_x > 0$ and the − path is taken if $B_x < 0$. For Energy, the minus path is taken upstream of B_x and the plus path is taken downstream of B_x (i.e. use the opposite sign to $B_x S_M$).

As with the outer star states, if `HLLD_PHYSICAL_SAFETY`, is enabled in `Config.h`, then DRAGON will revert to HLLE if the inner star state produces a nonphysical thermal pressure.

If the zero line falls between an alfven speed S^*_K and contact speed S_M, the flux is given by

$$F^{**}_K = F^*_K + S^*_K(U^{**}_K - U^*_K) \tag{2.41}$$

on the appropriate side.

Zero Normal B

When the interface-normal field vanishes, the Alfvén waves collapse onto the contact wave and the five-wave structure collapses to three regions. DRAGON handles this with a separate code path, using the same S_L, S_R, S_M as above but without the Alfvén-speed split.

Outer star states are given by

$$\rho^*_K = \frac{\varrho_K}{S_K - S_M} \tag{2.42a}$$

$$(v_x)^*_K = S_M \tag{2.42b}$$

$$(v_y)^*_K = v_y \tag{2.42c}$$

$$(\mathbf{B})^*_K = \mathbf{B}_K \left[\frac{S_K - v_{x,K}}{S_K - S_M}\right] \tag{2.42d}$$

$$E^*_K = E_K \left(\frac{S_K - v_{x,K}}{S_K - S_M}\right) + \frac{p^*_T S_M - p_{T,K} v_{x,K}}{S_K - S_M} \tag{2.42e}$$

No inner state is computed since $S^*_L = S_M = S^*_R$. In the exceptional case where $S_M = 0$ exactly, the flux is taken as the density-weighted average of the two one-sided fluxes $F = \frac{\sqrt{\rho^*_L}F^*_L + \sqrt{\rho^*_R}F^*_R}{\sqrt{\rho^*_L} + \sqrt{\rho^*_R}}$ to preserve left-right symmetry at that single point.

2.7.6 Roe (Hydrodynamics Only)

The Roe solver (Roe, 1981; Roe and Pike, 1984) linearizes the Euler flux Jacobian about the Roe-averaged state of Section 2.7.2, giving the five eigenvalues

$$\lambda = (v_{x,M} - a_M,\ v_{x,M},\ v_{x,M},\ v_{x,M},\ v_{x,M} + a_M) \tag{2.43}$$

which correspond to the left acoustic, entropy, two shear, and right acoustic modes.

The corresponding right eigenvectors $K_1, \ldots, K_5$ and wave strengths are

$$\alpha_1 K_1 = \frac{(p_R - p_L) - \rho_M a_M (v_{x,R} - v_{x,L})}{2a^2_M} \begin{pmatrix} 1 \\ \mathbf{v}_M - (a_M, 0, 0)^T \\ H_M - v_{x,M} a_M \end{pmatrix} \tag{2.44a}$$

$$\alpha_2 K_2 = (\rho_R - \rho_L) - \frac{p_R - p_L}{a_M^2} \begin{pmatrix} 1 \\ \mathbf{v}_M \\ \frac{1}{2}\mathbf{v}_M^2 \end{pmatrix} \tag{2.44b}$$

$$\alpha_3 K_3 = \rho_M (v_{y,R} - v_{y,L}) \begin{pmatrix} 0 \\ (0,1,0)^T \\ v_{y,M} \end{pmatrix}, \qquad \alpha_4 K_4 = \rho_M (v_{z,R} - v_{z,L}) \begin{pmatrix} 0 \\ (0,0,1)^T \\ v_{z,M} \end{pmatrix} \tag{2.44c}$$

$$\alpha_5 = \frac{(p_R - p_L) + \rho_M a_M (v_{x,R} - v_{x,L})}{2a_M^2} \begin{pmatrix} 1 \\ \mathbf{v}_M + (a_M, 0, 0)^T \\ H_M + v_{x,M} a_M \end{pmatrix} \tag{2.44d}$$

The flux is then computed by

$$F_{\text{Roe}} = \frac{1}{2}(F_L + F_R) - \frac{1}{2}\sum_{k=1}^{5} \alpha_k |\lambda_k| K_k. \tag{2.45}$$

However, the Roe method requires an entropy fix (Harten and Hyman, 1983). This is enabled by `Harten_Hyman` in `Config.h`. The entropy fix checks whether the left or right acoustic wave is a transonic rarefaction. If this is the case, the offending eigenvalue is replaced by an interpolated value between the fan's bounding speeds before the flux is assembled.

2.7.7 Riemann Solver Fallback Behaviour

Most Riemann solvers are not guaranteed to produce physically sound results. DRAGON provides mechanisms to adaptively pivoting to a different solver should one fail. One such mechanism is the `HLLD_PHYSICAL_SAFETY` pathway described in Section 2.7.5, which causes the HLLD solver to switch to HLLE in the event that a star state pressure is nonphsyical.

The other pathway is enabled via `RIEMANN_VERIFY_FALLBACK` in `Config.h`. In this method, the states $w_L - \xi \frac{\Delta t}{\Delta x} F_{i+\frac{1}{2}}$ and $w_R + \xi \frac{\Delta t}{\Delta x} F_{i+\frac{1}{2}}$ are calculated for $\xi =$`CONFIG::Riemann_ExactFallback_Parameter`. If either of the resulting quantities is nonphysical, the following fallback procedure is used

1. Recalculate the flux using HLLE (see Section 2.7.3)
   (a) If this result passes the above physicality test, return it
   (b) If this result fails the above physicality test, proceed to step 2
   (c) This step can be skipped by disabling `RIEMANN_FALLBACK_TRY_HLLE` in `Config.h`.
2. In Hydrodynamic mode, recalculate the flux using the Exact solver (see Section 2.7.1)
   (a) If this result passes the above physicality test, return it
   (b) If this result fails the above physicality test, proceed to step 3
   (c) In MHD mode, proceed directly to step 3.
3. Throw an exception, which will typically result in the entire timestep being restarted.

From our own testing we have found that `HLLD_PHYSICAL_SAFETY` is a more robust fallback pattern for the HLLD solver, and as such recommend disabling `RIEMANN_VERIFY_FALLBACK` for MHD builds.

2.8 Conservative Updates to Primitive States

Several of the methods thus far require implementing an equation of the form $U' = U + \Delta U$, where U_i are states and ΔU is a change in conserved quantities. However, since states U and U' are often stored in primitive forms W and W', care must be taken to avoid floating point errors from a round-trip conversion. Since density and magnetic field are the same in both primitive and conservative form, these quantities can be updated as $\rho' = \rho + \Delta\rho$ and $\mathbf{B}' = \mathbf{B} + \Delta\mathbf{B}$ respectively. To update velocity, we can use

$$\mathbf{v}' = \mathbf{v} + \frac{1}{\rho}\left[\mathbf{f} - \mathbf{v}\Delta\rho\right] \tag{2.46}$$

where $\mathbf{f}$ is the change in momentum density $\rho\mathbf{v}$. If both $\mathbf{f}$ and $\Delta\rho$ are zero, then $\mathbf{v}$ will be exactly unchanged. Meanwhile, for the pressure update , we derive from equation 2.2 the result

$$p' = p + (\gamma - 1)\left[\Delta E - \frac{v^2}{2}\Delta\rho - \rho'\left(\frac{\mathbf{v}' + \mathbf{v}}{2}\right)\cdot\Delta\mathbf{v} - \left(\frac{\mathbf{B}' + \mathbf{B}}{8\pi}\right)\cdot\Delta\mathbf{B}\right] \tag{2.47}$$

where ΔE is the change in energy density, $\Delta\mathbf{v} = \mathbf{v}' - \mathbf{v}$ is as given by equation 2.46. Updating pressure in this way ensures that if all components of ΔU are zero, then all components of W will remain unchanged at a bit level. As an added benefit, this procedure requires fewer floating point multiplications than a round trip conversion to conservative state.

In DRAGON, this procedure is implemented in the `PrimitiveState += ConservativeState` operator.

2.9 Constrained Transport

DRAGON uses constrained transport (Evans and Hawley, 1988) to ensure that magnetic fields remain free of divergences. The specific implementation is largely based on Gardiner and Stone (2008).

2.9.1 Computation of Electric Fields

Because the induction equation is part of the same conservative MHD system solved by the Riemann solvers of Section 2.7, the resulting fluxes already contain the electric field (up to a sign) via the transverse-B components. Specifically, the flux of B^a through an b-face is $\mp E^c$, where the minus (plus) sign is taken when abc a cyclic (anticyclic) permutation of xyz. DRAGON extracts this information directly via `CT::computeElectric`, rather than reconstructing $\mathbf{E}$ separately.

In 3D, each edge-centered component of $\mathbf{E}$ touches four adjacent faces, and is taken as their average:

$$E^x_{i,j-\frac{1}{2},k-\frac{1}{2}} = \frac{1}{4}\left[F^{Z,B_y}_{i,j-1,k-\frac{1}{2}} + F^{Z,B_y}_{i,j,k-\frac{1}{2}} - F^{Y,B_z}_{i,j-\frac{1}{2},k-1} - F^{Y,B_z}_{i,j-\frac{1}{2},k}\right] \tag{2.48a}$$

$$E^y_{i-\frac{1}{2},j,k-\frac{1}{2}} = \frac{1}{4}\left[F^{X,B_z}_{i-\frac{1}{2},j,k-1} + F^{X,B_z}_{i-\frac{1}{2},j,k} - F^{Z,B_x}_{i-1,j,k-\frac{1}{2}} - F^{Z,B_x}_{i,j,k-\frac{1}{2}}\right] \tag{2.48b}$$

$$E^z_{i-\frac{1}{2},j-\frac{1}{2},k} = \frac{1}{4}\left[F^{Y,B_x}_{i-1,j-\frac{1}{2},j,k} + F^{Y,B_x}_{i,j-\frac{1}{2},k} - F^{X,B_y}_{i-\frac{1}{2},j-1,k} - F^{X,B_y}_{i-\frac{1}{2},j,k}\right] \tag{2.48c}$$

where $F^{X,B_y}_{i-\frac{1}{2},j,k}$ denotes the B^y component of the flux returned by the x-direction Riemann solve at face $(i-1/2, j, k)$, and similarly for the other flux/component pairs. In 2D, the E_z component is likewise derived as

$$E^z_{i-\frac{1}{2},j-\frac{1}{2}} = \frac{1}{4}\left[F^{Y,B_x}_{i-1,j-\frac{1}{2},j} + F^{Y,B_x}_{i,j-\frac{1}{2}} - F^{X,B_y}_{i-\frac{1}{2},j-1} - F^{X,B_y}_{i-\frac{1}{2},j}\right]. \tag{2.49}$$

while the E_x and E_y components only touch one face:

$$E^x_{i,j-\frac{1}{2}} = -F^{Y,B_z}_{i,j-\frac{1}{2}}, \qquad E^y_{i-\frac{1}{2},j} = F^{X,B_z}_{i-\frac{1}{2},j} \tag{2.50}$$

The above four-face average is only first-order accurate at the edge and can allow spurious checkerboard-like noise to grow. DRAGON corrects this in `CT::upwindElectric` using the unsplit Corner Transport Upwind (CTU) scheme of Gardiner and Stone (2008).

Each edge-centred **E** component is upwinded once per adjacent face, using the sign of the mass flux through that face to choose which side to draw from:

$$\text{upwindCorr}(\dot{m}, E^{\text{face}}_-, E^{\text{face}}_+, E^{\text{body}}_-, E^{\text{body}}_+) = \begin{cases} \frac{1}{4}\left(E^{\text{face}}_- - E^{\text{body}}_-\right) & \dot{m} > \epsilon, \\ \frac{1}{4}\left(E^{\text{face}}_+ - E^{\text{body}}_+\right) & \dot{m} < -\epsilon, \\ \frac{1}{8}\left(E^{\text{face}}_- + E^{\text{face}}_+ - E^{\text{body}}_- - E^{\text{body}}_+\right) & |\dot{m}| \le \epsilon, \end{cases} \tag{2.51}$$

with $\epsilon = 10^{-18}$ (`CTU_TOL` in `Electric.cpp`) guarding the near-zero-velocity case. Here $\dot{m}$ is the mass flux[7] through the relevant face. $E^{\text{body}}_\pm$ are the cell-centred reference values $\mathbf{E} = -\mathbf{v} \times \mathbf{B}$ (calculated by `CT::bodyElectric`) in the two cells adjacent to that face, and $E^{\text{face}}_\pm$ are the face-centred field values values (as used in `CT::computeElectric`) in the faces which 1) border both the cell and edge in question and 2) are not the face through which mass flux is taken. Each edge component receives one such correction per bordering face, and the corrections are summed onto the flux-averaged value from `CT::computeElectric`.

2.9.2 Faraday's Law

Given the finalized upwinded edge **E**, the face-normal B-fields are advanced using

$$\frac{\partial \mathbf{B}}{\partial t} = -\nabla \times \mathbf{E}. \tag{2.52}$$

`CT::Faraday` evaluates this curl such that each face component only involves the edge E components that actually bound it. In 3D, this is

$$\Delta B^x_{i-\frac{1}{2},j,k} = \frac{\Delta t}{\Delta z}\left[E^y_{i-\frac{1}{2},j,k+\frac{1}{2}} - E^y_{i-\frac{1}{2},j,k-\frac{1}{2}}\right] - \frac{\Delta t}{\Delta y}\left[E^z_{i-\frac{1}{2},j+\frac{1}{2},k} - E^z_{i-\frac{1}{2},j-\frac{1}{2},k}\right] \tag{2.53a}$$

$$\Delta B^y_{i,j-\frac{1}{2},k} = \frac{\Delta t}{\Delta x}\left[E^z_{i+\frac{1}{2},j-\frac{1}{2},k} - E^z_{i-\frac{1}{2},j-\frac{1}{2},k}\right] - \frac{\Delta t}{\Delta z}\left[E^x_{i,j-\frac{1}{2},k+\frac{1}{2}} - E^x_{i,j-\frac{1}{2},k-\frac{1}{2}}\right] \tag{2.53b}$$

$$\Delta B^z_{i,j,k-\frac{1}{2}} = \frac{\Delta t}{\Delta y}\left[E^x_{i,j+\frac{1}{2},k-\frac{1}{2}} - E^x_{i,j-\frac{1}{2},k-\frac{1}{2}}\right] - \frac{\Delta t}{\Delta x}\left[E^y_{i+\frac{1}{2},j,k-\frac{1}{2}} - E^y_{i-\frac{1}{2},j,k-\frac{1}{2}}\right] \tag{2.53c}$$

The corresponding 2D update follows the same pattern for the two in-plane face fields B^x, B^y (using E^z only) and the cell-centered B^z (using both E^x and E^y). This update, by construction, leaves $\nabla \cdot \mathbf{B}$ unchanged from its value before the step.

[7] Gardiner and Stone (2008) use velocity, but mass flux will almost always have the same sign and fluxes are already being passed to `CT::upwindElectric`

2.9.3 Calculation of Body-Centred Fields

Once the face-centred normal fields are calculated, the body-centred fields are calculated as follows

$$B^x_{i,j,k} = \frac{1}{2}\left(B^x_{i-\frac{1}{2},j,k} + B^x_{i+\frac{1}{2},j,k}\right) \tag{2.54a}$$

$$B^y_{i,j,k} = \frac{1}{2}\left(B^y_{i,j-\frac{1}{2},k} + B^y_{i,j+\frac{1}{2},k}\right) \tag{2.54b}$$

$$B^z_{i,j,k} = \frac{1}{2}\left(B^z_{i,j,k-\frac{1}{2}} + B^z_{i,j,k+\frac{1}{2}}\right) \tag{2.54c}$$

However, a complication arises from the magnetic contribution to energy. Unless the updated value happens to match the previous value exactly, it isn't possible to keep both total and thermal energy constant through the CT-update. DRAGON supports three different procedures for handling the energy in the CT update:

- *Always keep total energy constant* (`CT_CONSV_TOTAL_E`). This is done by converting $\mathbf{w}$ to conservative form, setting $\mathbf{B}$ on the conservative form, and then converting back to primitive form. This method ensures energy conservation, but may be prone to numerical issues where thermal pressure irrecoverably becomes nonphysical.

- *Always keep thermal pressure constant* (`CT_CONSV_THERMAL`). This is done by simply setting $\mathbf{B}$ on the primitive state $\mathbf{w}$. This method is the most robust, but will almost always violate energy conservation. This may be justifiable in some astrophysical flows on grounds that radiation violates energy conservation anyway.

- *Dynamically switch between the above options* (`CT_CONSV_BETA_GATED`). In this procedure, the local value of the plasma beta $\beta_m = \frac{8\pi p}{\mathbf{B}^2}$ is calculated and compared to `CONFIG::CT_Energy_Beta`. For cells with a plasma beta below the threshold, thermal energy is conserved to avoid numerical issues; otherwise, total energy is conserved. This option allows the simulation to remain closer to total energy conservation while still avoiding numerical issues.

The desired option can be selected by setting `CT_ENERGY_CONSV` in `Config.h`.

2.10 Passive Scalars

Passive scalars are quantities which are advected by the fluid but don't feed back into its evolution. Passive scalars have a number of uses, such as tracking chemical composition or the origin of a particular patch of fluid. The governing equation for each passive scalar q_i is

$$\frac{\partial}{\partial t}(\rho q_i) + \nabla \cdot (\rho \mathbf{v} q_i) = 0 \tag{2.55}$$

At each interface, the flux of each passive scalar is calculated by multiplying the mass flux by q_i from the upwind cell.

To add a passive scalar to your grid, call `grid.passives().add(key_str)`, where `key_str` is a string used to identify the name of the scalar. In addition to identifying the scalar within DRAGON, this key is used to denote the scalar within output files and can therefore be used to plot the scalar in DRAGONGAZE. If you no longer need a particular scalar and wish to free its memory, you can remove it by calling `grid.passives().remove(key_str)`. Attempting to add a key which is already there or removing a nonexistent key won't do anything.

The current values of the passive at each cell can be accessed via `grid.passives()[i, key_str]`, `grid.passives()[i,j, key_str]`, or `grid.passives()[i,j,k, key_str]` as appropriate for the dimension. Alternatively, `grid.passives()[i]`, `grid.passives()[i,j]`, or `grid.passives()[i,j,k]` will return a vector containing all of the passive values at a given cell, which can later be disambiguated by calling `grid.passives().lookup(passive_vector, key_str)` if needed.

Because passive scalars are by definition passive, only one layer of ghost cells is stored. For outflow and reflective boundaries, each cell in the ghost layer is filled by the immediately adjacent physical cell; for periodic boundaries, the ghost cells instead match the opposite side. Fixed-state boundary conditions don't define how the boundary should behave for passive scalars, you would need to implement that yourself.

Chapter 3

Source Terms

Chapter 4

Performance: DRAGONWING

DRAGON Wide Infrastructure Numerical Grids (DRAGONWING) is DRAGON's support library for memory management and multithreading.

4.1 Memory Management

DRAGON requires scratch arrays to store the information computed in the Reconstruction (Section 2.6), flux calculation (Section 2.7), and constrained transport (Section 2.9) stages. These scratch arrays are of the same size as the main grid, though the CT scratch arrays only hold 3-vectors instead of the full 8-component fluid states. Rather than have every call site `new`/`delete` these buffers, DRAGONWING centralises allocation behind three request functions, overloaded for 1D/2D/3D grids:

- `requestPrimitiveArrays(N, ...)` – returns `N` scratch arrays of `PrimitiveState`
- `requestFluxArrays(N, ...)` – returns `N` scratch arrays of `ConservativeState` (used to store fluxes)
- `requestVec3Arrays(N, ...)` – returns `N` scratch arrays of `vec3` (used e.g. for electric fields).

Each of these returns a `DRAGONWING::ArrayGuard`, a small RAII wrapper templated on the underlying `ExtendedArray` type. `ArrayGuard` provides a collection of raw pointers to the allocated arrays, which are returned to DRAGONWING upon destruction of the `ArrayGuard`. Arrays reclamation happens automatically when the guard goes out of scope (exception-safely included), or can be done early by calling `guard.release()`. Note that it is the responsibility of the caller to cease all use of any arrays allocated by the guard once `release()` is called.

`REUSE_AUX_GRIDS` is enabled (this is the default) in `DRAGONWING_Config.hpp`, DRAGONWING keeps a static pool of previously-allocated arrays for each combination of element type (`PrimitiveState`/`ConservativeState`/`vec3`) and dimensionality (1D/2D/3D). Each of the three element types has its own `std::mutex` (`pm`, `cm`, `vm`), so pooling is thread-safe. Each request for new arrays first scans the corresponding pool for inactive arrays whose size match the request. Any such arrays are then marked as active and reused as is. Only if the existing arrays are insufficient to meet the caller's request are new arrays allocated with `new`; these arrays are then added to the pool and marked as active. When an array is released (via `ArrayGuard::release()` it is marked inactive again; the memory is not freed and is instead available for reuse. Because pooled arrays can carry stale data from a previous use, any code that requests scratch arrays must treat their contents as unspecified before writing to them. DRAGONWING makes no guarantee of zero-initialization. Arrays are only deleted when the program terminates, or if `DRAGONWING::purgeAllBuffers()` is called. This function

frees every array currently sitting inactive in the pools via `delete`, though Active arrays are left alone so as to not interfere with the currently executing process; these arrays will instead be freed individually the next time they are released back to DRAGONWING.

If instead `REUSE_AUX_GRIDS` is disabled, Requests for arrays always allocate arrays with `new`, and releasing arrays always frees the array with `delete`. No pool is kept, so there is nothing to purge and no stale-data concern, at the cost of paying full allocation/deallocation overhead on every step.

4.2 Multithreading

`DRAGONWING::ThreadPool` is used by `DistGrid` (see Section 4.3) to advance sibling subgrids concurrently, each on its own thread. It is constructed with a fixed thread budget `N` (one thread per child subgrid); To advance by one advection step, `launchParallel(grid, dt)` is called once per child to start `grid->advance_step(dt)` running on a new thread; for strang-split source terms, `sourceParallel(src,grid,dt)` is called once per child to start `src->advance(*grid, dt)`. In either case, once advancement of all children has been initiated, call `waitForCompletion()` on the `ThreadPool` to wait for all children to complete their work. `waitForCompletion()` returns `true` if all children advanced successfully; if `false` is returned, the step will need to be restarted.

4.2.1 Checkpoint Protocol

From a synchronisation standpoint, each advection-step advancement of a subgrid is divided into two phases:

1. Each subgrid computes all updated values and verifies that the solution is physically admissible (steps 1-10 in Section 2.4.2). The original array remains in tact so that the step can be restarted if necessary. In an unsplit advection step, this phase uses a significant amount of scratch memory.

2. Each subgrid commits the preliminary update to the base array (step 12 in Section 2.4.2).

All subgrids must complete phase 1 before any can enter phase 2. This requirement allows any subgrid to request that the step be restarted before the existing data is overwritten[1].

Subgrids maintain these synchronisation phases as follows:

1. `DRAGONWING::waitForRelease()` (Optional but recommended). Since phase 1 typically uses a significant amount of scratch memory, the user might optionally set `DRAGONWING::CONFIG::phase_1_max_threads` in `DRAGONWING_Config.hpp` to limit program's total memory usage. The `ThreadPool` will track the number of threads currently in phase 1. If the number of threads is already at its limit, the caller will be blocked until another thread either reports completion of phase 1 or requests a step restart. `waitForRelease()` returns a boolean, which is true if the caller is clear to proceed with phase 1 and false if another subgrid has requested a restart (in which case the calling function should return instead of proceeding).

2. Do the work of phase 1.

   (a) If an error occurs, throw a `std::exception`. `launchParallel`'s exception handler will then call `DRAGONWING::requestRestart(error_msg)` to request that all subgrids restart the current step. For the first subgrid to request a step restart, the `error_msg` string can be accessed via `ThreadPool.restartMsg()`. Calling `restartMsg()` will then clear the message.

[1] As of version 1.2 this is no longer strictly necessary given the addition of `Grid::backup` and `Grid::restore`, but the protocol remains in place for the time being

3. `DRAGONWING::reportChckpoint1()`. This informs the local `ThreadPool` that another subgrid has completed phase 1. This will wake up any threads currenlty blocked in `waitForRelease()` and will allow one to proceed, or if it is the last function to complete it will wake up all threads blocked by in the next step

4. `DRAGONWING::waitForCheckpoint1()`. This will block the current thread until all subgrids have completed phase 1. This function returns a boolean, which is true if the caller is clear to proceed with phase 2 and false if another subgrid has requested a restart (in which case the calling function should return instead of proceeding).

5. Do the work of phase 2

6. `DRAGONWING::reportChekcpoint2()`. This informs the thread pool that this subgrid has completed its work.

The implementations of these functions lives in the `ThreadPool` class, but the caller need not know which `ThreadPool` is in use. DRAGONWING maintains a `thread_local` variable, allowing the `DRAGONWING ::` functions to call the corresponding function on the `ThreadPool` that created the thread from which it is called.

After all subgrids are launched via `launchParallel(grid, dt)`, call `waitForCompletion()` on the same `ThreadPool` object. This will wait until either either all threads have reported completion of phase 2 or any thread has requested a step restart. `waitForCompletion()` will return true if the step was successful, or false if it needs to be restarted.

4.3 Domain Decomposition

`DRAGON::DistGrid<T>` implements domain decomposition, in which a larger gird is split into smaller subgrids which can be advanced concurrently on separate threads. Although this class is found in `DRAGON/Refinement`[2] instead of `DRAGONWING/` proper, it is documented in this chapter because it is DRAGON's only current form of parallelism and depends directly on the `ThreadPool`/checkpoint machinery of Section 4.2.

`DistGrid<T>` is a CRTP-style base combined with `Grid1D`/`Grid2D`/`Grid3D` by multiple inheritance to form `DistGrid1D`, `DistGrid2D`, and `DistGrid3D`. Each holds a `std::vector<std::unique_ptr<T>>` of children – subgrids of the same concrete type. The tree can in principle nest to arbitrary depth, but until such time that AMR is implemented DRAGON will in practice only ever build to one level: a parent (constructed with `root=true`) and its children (constructed with `root=false`). A grid with `children.size() <= 1` is treated as a leaf and steps itself directly rather than delegating to its (nonexistent or single) children.

4.3.1 Load Balancing

The number of children along each axis is chosen to keep child subgrids close to square (2D) or cubic (3D), so that surface-to-volume overhead from ghost-cell synchronization is not concentrated in one direction:

- **1D:** `ncx = CONFIG::core_count` directly – every core gets one child.

[2]DRAGON does not yet support Adaptive Mesh Refinement (AMR), but the `Refinement` directory uses this name in anticipation of future AMR implementation since Domain Decomposition is required for AMR. When such implmentation is added, this section will likely move to the AMR chapter.

- **2D:** solving $n_x/n_{cx} = n_y/n_{cy}$ and $n_{cx}n_{cy} \gtrsim$ core_count gives

$$n_{cx} = \left\lceil \sqrt{\text{core_count} \cdot \tfrac{n_x}{n_y}} \right\rceil, \qquad n_{cy} = \left\lceil \sqrt{\text{core_count} \cdot \tfrac{n_y}{n_x}} \right\rceil. \tag{4.1}$$

- **3D:** the analogous cube-root balancing is used for n_{cx}, n_{cy}, n_{cz}, giving

$$n_{cx} = \left\lceil \left(N \cdot \frac{n_x^2}{n_y n_z} \right)^{\frac{1}{3}} \right\rceil, \qquad n_{cy} = \left\lceil \left(N \cdot \frac{n_y^2}{n_x n_z} \right)^{\frac{1}{3}} \right\rceil, \qquad n_{cy} = \left\lceil \left(N \cdot \frac{n_z^2}{n_x n_y} \right)^{\frac{1}{3}} \right\rceil, \tag{4.2}$$

  where N is equal to `CONFIG::core_count`.

Because each factor is rounded up independently, the product $n_{cx}n_{cy}(n_{cz})$ is not guaranteed to equal `core_count` exactly – DRAGON may use somewhat more subgrids (and hence threads) than `core_count` when the aspect ratio does not divide evenly.

Given a child count n_c along an axis, `computeChildSize(nx, nc, i)` distributes the n_x cells as evenly as possible: every child gets `nx / nc` cells, and the first `nx % nc` children (by index) each get one extra cell, so no two children differ in size by more than one cell.

4.3.2 Ghost Cells

Child subgrids need enough ghost cells to reconstruct correctly at their boundaries without an intra-step resynchronization with their siblings. `validGhosts(g)` enforces a scheme-dependent minimum, taking the larger of the caller's requested depth `g` and the required minimum number of ghosts. In general, a grid needs the following number of ghosts

- If MUSCL reconstruction is disabled, only a single ghost (the boundary) is required.
- If MUSCL reconstruction is enabled, a second ghost is required in order to reconstruct the boundary state correctly.
- For unsplit (CTU) integration with MHD, a third ghost is requried. This is because constrained transport needs the first layer of transverse fluxes in the boundary, and with CTU integration these fluxes are effected transverse corrections from the third layer of ghosts from the boundary.

However, for split integration, twice as many ghosts are needed as a result of each step performing two half-steps along one or more dimensions.

Because a child subgrid's ghost cells are filled directly by its parent rather than through the standard boundary-condition API, every child is constructed with `boundary = Boundary::Ignore()` (Section 2.3).

4.3.3 Parallel Step & Synchronization

`DistGrid<T>::step(dt)` is the actual parallel execution, called whenever a `split_step`/`unsplit_step` is invoked on a grid that has more than one child:

1. `pushToChildren()` copies the parent's fluid state (including its own ghost layer) into each child, offset appropriately for that child's position in the domain, then recurses into `child ->pushToChildren()` so any grandchildren would be synchronised too.

2. Construct a `DRAGONWING::ThreadPool` sized to the number of children, and call `launchParallel( child, dt)` for each one, launching its `split_step`/`unsplit_step` on its own thread.

3. `waitForCompletion()` blocks until every child has reported checkpoint 2 (Section 4.2) or a restart is requested.

4. If any child failed, throw a `std::runtime_error` carrying the pool's restart message – this propagates up into the same retry-with-halving substep loop as any other step failure (Section 2.5).

5. Otherwise, `loadFromChildren()` folds the children's updated interiors back into the parent. This function copies back only the child's interior cells (no ghosts) into the corresponding region of the parent. It also calls each child's `loadFromChildren()` prior to folding the child's data to ensure any grandchildren would be synchronised too.

Leaf grids (`children.size() <= 1`) bypass all of this: their `split_step`/`unsplit_step` simply calls the corresponding base `Grid1D/2D/3D` method directly and reports checkpoint 2 itself, since there are no children to launch or wait on. `on_step_fail()` follows the same split: a leaf delegates to the base `Grid`'s recovery logic, while a parent (with more than one child) always returns `true` – there is no grid further up the decomposition to hand restart responsibility to, so the parent itself must trigger the retry.

4.4 Atomics

DRAGONWING is responsible for managing any atomic registers which need to be accessed across multiple threads. Currently, there is one such register, which tracks fallback behaviours. When `DRAGONWING::reportFallback(weight,threshold)` is called, the fallback counter is incremented by `weight`. If this causes the counter to attain a value greater than `threshold`, an exception will be thrown to trigger a restart of the current step. The counter is then reset by calling `DRAGONWING::resetFallbacks()`.

DRAGON reports fallback in the following cases

- When `Riemann::verify_and_fallback` (Section 2.7.7) detects the initial solution to be unphysical.
- When HLLD computes a nonphysical star state and reverts to HLLE
- When MUSCL computes a nonphysical interface & reverts to piecwise-constant reconstruction
- If `CT_CONSV_BETA_GATED` is enabled, when the local plasma beta is less than `Config::ct_energy_beta` and thermal pressure is protected.

The weights for these fallbacks are defined in DRAGON's `Config.h`: `Config::fallback_weight_riemann` for the first two cases, `Config::fallback_weight_MUSCL` for MUSCL falling back to PCM reconstruction, and `Config::fallback_weight_ct_beta` for the low-beta guard cases.

Each of the fallback cases will also pass `Config::fallback_limit` as the threshold for triggering a restart. This limit is the upper bound of the safes zone, it must be *exceeded* to trigger a step restart. Note that `fallback_limit` don't take resolution into account; if you are running the same simulation at multiple resolutions, you might consider increasing `fallback_limit` as resolution increases.

As an example, if we use

```
constexpr int fallback_limit = 6;
constexpr int fallback_weight_riemann = 3;
constexpr int fallback_weight_MUSCL = 4;
constexpr int fallback_weight_ct_beta = 0;
```

then a restart will be triggered after three Riemann solver fallbacks, two MUSCL fallbacks, or one of each (plasma-beta fallbacks will not trigger step restarts in this example).

Chapter 5

File Output: DRAGONHOARD

DRAGON Hierarchical Output for Analysis, Restarts, and Diagnostics (DRAGONHOARD) is DRAGON's I/O library: it writes simulation output, and supports loading them back in as restart-checkpoints. The public interface (`DragonHoard.hpp`) exposes `writeToFile`/`loadFromFile`, each overloaded once per grid dimensionality plus a `Grid`-dispatching version that picks the right overload with a `dynamic_cast`.

Each frame is written as a single HDF5 file named `<output_base_name>_#####.h5`, where the five-digit (zero-padded) cycle number comes from `cycle_string()` and `output_base_name` is set in the module's configuration header `DRAGONHOARD_Config.h`. DRAGONHOARD automatically appends the `.h5` extension to a bare filename when it is not already present, so callers of `writeToFile`/`loadFromFile` do not need to include it themselves.

The output directory `output_dir` is also set in `DRAGONHOARD_Config.h`. `verifyOutputDirectory()` creates `output_dir` if it does not already exist (and throws if the path exists but is not a directory); it is expected to be called once before the first write of a run (the default DRAGON `main.cpp` does this).

5.1 File Format

Every array is written gzip-compressed at level `HDF5_COMPRESSION_LEVEL` (set in `DRAGONHOARD_Config.h`, nonpositive to disable), chunked at up to 1024 cells (1D), 32×32 (2D), or $16 \times 16 \times 16$ (3D). Chunking is capped to the array's own extent on small grids.

Each file also carries a small set of scalar attributes describing its contents: `format_version`, `write_option`, `dim`, `MHD`, `nx`/`ny`/`nz`, `dx`/`dy`/`dz`, `cycle`, and `time`. `ng` contains the ghost depth actually written, though this is normally zero[1] since ghosts are normally regenerated by the boundary conditions (Section 2.3). 2D/3D MHD files additionally store `B_floats`, recording the precision of the recorded body-centred fields.

Density (`rho`) is always written as are face-normal B fields `Bf` (if MHD is enabled). The other components depend on `HDF5_WRITE_OPTION` and `HDF5_REDUNDANT_VALS_OPTION`.

Which variables are written is controlled by `HDF5_WRITE_OPTION` in `DRAGONHOARD_Config.h`. This bitmask is as follows:

- bit 1 – primitive variables (`v`, `p`)
- bit 2 – energy density (`E`)
- bit 4 – conserved variables (`mom`, `E`)

[1] If you do need write ghosts for debugging purposes, enable `WRITE_GHOSTS_TO_FILE` in `DRAGONHOARD_Config.h`

The following presets are given: `HDF5_WRITE_PRIMITIVE` (1), `HDF5_WRITE_PRIMITIVE_AND_ENERGY` (3, the default), `HDF5_WRITE_CONSERVATIVE` (6), and `HDF5_WRITE_PRIMITIVE_AND_CONSERVATIVE` (7, both forms, at the cost of a larger file). A `#error` in `HDF5Output.cpp` rejects any option that sets neither the primitive nor the conservative bit, since that would leave the fluid state under-determined.

Body-centred `B` in MHD builds is written whenever `HDF5_REDUNDANT_VALS_OPTION` is not `HDF5_WRITE_OMIT`; that same setting also controls whether it (and, when the energy bit is set without the momentum bit, `E`) is stored as `float` or `double`. These values can be recomputed by loader, so trading precision for file size is safe in this case.

5.2 Restart and Loading

Loading mirrors writing: `loadFromFile` dispatches to the matching `Grid1D`/`Grid2D`/`Grid3D` overload and opens the file read-only. Before touching the grid it checks compatibility, rejecting a file with a newer `format_version` than this build understands, a mismatched dimensionality or dimensions[2], or (in a hydro build) a file that was written with MHD enabled.

Density and (in MHD builds) magnetic fields are loaded independent of `write_option`. The remaining load is determined by the file's `write_option`, not the current build's `HDF5_WRITE_OPTION` setting. If the file contains the full primitive state, then velocity and pressure are read; only if the file is conservative-only are energy and momentum read and subsequently converted to primitive states. Finally, if the body-centred `B` was not stored at full precision (`B_floats` $\neq$ `HDF5_WRITE_DOUBLE`), the loader discards whatever it read and calls `grid.initialize_B_fields()` to recompute it exactly from the (always double-precision) face field.

Whether a run attempts to restart at all is controlled at compile time by `RESTART_FROM_FILE` in `DRAGONHOARD_Config.h`. When restarts are enabled, `RESTART_FRAME` selects which frame to load; if negative (the default), `restartFileName()` scans `output_dir` and restarts from whichever `..._#####.h5` file has the highest cycle number.

[2] `nx`/`ny`/`nz` must match exactly, there is currently no support for restarting onto a differently sized grid.

Chapter 6

Running DRAGON

This chapter covers configuring, building, and running DRAGON itself. For turning simulation output into plots once a run has completed, see Chapter 7.

6.1 Configuration

Most numerical choices are compile-time settings in `DRAGON/Config.h`. The most important switches are:

- `MHD` – enables magnetohydrodynamics. Disable for pure hydrodynamics.
- `DIMENSION_UNSPLIT` – selects unsplit multidimensional advancement (disable to instead use Strang splitting).
- `RIEMANN_DEFAULT` – the default Riemann solver (Section 2.7). The default `RIEMANN_HLLX` uses HLLC for pure hydrodynamics and HLLD for MHD.
- `CFL_CALCULATION` – chooses the CFL speed calculation method for multidimensional problems (Section 2.5).
- `CFL_coeff` – the global CFL coefficient (Section 2.5); the default value is 0.3.
- `MUSCL_DEFAULT_LIMITER` – the TVD limiter used by MUSCL reconstruction.
- `core_count` – how many child grids the root should create when using DistGrid (Section 4.3).

Physical parameters are instead found in `DRAGON/Constants.h`. Currently the only such value is the specific heat ratio `_gamma`, which defaults to 5/3.

6.2 Setting Up a Problem

Along with `Config.h`, `Problem.cpp` contains the information needed to set up the problem you wish to run: both files are filled with comments explaining how to configure a particular problem. To run a problem, edit `Config.h` to choose the numerical options (Section 6.1) and edit `Problem.cpp` to define the initial conditions (Section 2.2), boundary conditions (Section 2.3). Lastly, configure `DRAGONHOARD/DRAGONHOARD_Config.hpp` with your desired output settings (Chapter 5).

The `Examples/` folder includes several ready-made example problems. To run one of these problems, replace `Config.h`, `Problem.cpp`, and (if applicable) `Constants.h` with the desired example's versions of those files.

6.3 Building on macOS with Xcode

In Xcode, open the project's build settings and add the following, using the path to your HDF5 installation (Section 1.3) in place of `<Path_To_HDF5>`:

- `<Path_To_HDF5>/include` to *System Header Search Paths*
- `<Path_To_HDF5>/lib` to *Library Search Paths*
- `<Path_To_HDF5>/lib` to *Runpath Search Paths*
- `-lhdf5_cpp` to *Other Linker Flags*
- `-lhdf5` to *Other Linker Flags*

Once this is done, you should be able to build and run the DRAGON target, or the DRAGON_TESTS target for the unit test suite.

If the project builds but fails when running, open *Signing & Capabilities* for each executable target and ensure *Disable Library Validation* is enabled under *Hardened Runtime*. After changing this setting, clean and rebuild before running again.

6.4 Building from Command Line

6.4.1 Creating the bin Directory

The first time you build DRAGON from the command line, you will need to create the 'bin/' directory that the build output will go into. From the project's root directory, run:

```
mkdir -p bin
```

The entire bin directory is ignored by git, so anything you put in there won't be committed to the repository. This is intended to avoid putting executables in the repository.

6.4.2 Building on macOS

From the project's root directory, run:

```
HDF5_PATH=$(brew --prefix hdf5)
clang++ -std=c++23 -O3 \
    -o bin/DRAGON \
    -I"$HDF5_PATH/include" \
    -L"$HDF5_PATH/lib" \
    -Wl,-rpath,"$HDF5_PATH/lib" \
    -IDRAGON -IDRAGONWING -IDRAGONHOARD \
    $(find DRAGON DRAGONWING DRAGONHOARD -name "*.cpp") \
    -lhdf5_cpp -lhdf5
```

From here, you can run DRAGON by calling

```
bin/DRAGON
```

or you can copy the executable to another directory and run it there.

The test suite can be built similarly as follows:

```
HDF5_PATH=$(brew --prefix hdf5)
clang++ -std=c++23 -O3 \
    -DTESTMODE \
    -o bin/DRAGON_TESTS \
    -I"$HDF5_PATH/include" \
    -L"$HDF5_PATH/lib" \
    -Wl,-rpath,"$HDF5_PATH/lib" \
    -IDRAGON -IDRAGONWING -IDRAGONHOARD -ITesting\
    $(find DRAGON DRAGONWING DRAGONHOARD Testing -name "*.cpp") \
    -lhdf5_cpp -lhdf5
```

6.4.3 Building on Linux

You can compile with your normal C++ compiler and manually provide the HDF5 include and library paths, along with DRAGON's own header search paths. If HDF5 is installed in a standard system location, this may be sufficient:

```
g++ -std=c++23 -O3 \
    -o bin/DRAGON \
    -IDRAGON -IDRAGONWING -IDRAGONHOARD \
    $(find DRAGON DRAGONWING DRAGONHOARD -name "*.cpp") \
    -lhdf5_cpp -lhdf5
```

The test suite can be built similarly

```
g++ -std=c++23 -O3 \
    -DTESTMODE \
    -o bin/DRAGON_TESTS \
    -IDRAGON -IDRAGONWING -IDRAGONHOARD -ITesting \
    $(find DRAGON DRAGONWING DRAGONHOARD Testing -name "*.cpp") \
    -lhdf5_cpp -lhdf5
```

If your system provides the HDF5 compiler wrapper `h5c++`, it can be used instead of specifying the linker flags by hand:

```
h5c++ -std=c++23 -O3 \
    -o bin/DRAGON \
    -IDRAGON -IDRAGONWING -IDRAGONHOARD \
    $(find DRAGON DRAGONWING DRAGONHOARD -name "*.cpp")
```

```
h5c++ -std=c++23 -O3 \
    -DTESTMODE \
    -o bin/DRAGON_TESTS \
    -IDRAGON -IDRAGONWING -IDRAGONHOARD -ITesting \
    $(find DRAGON DRAGONWING DRAGONHOARD Testing -name "*.cpp")
```

If the compiler cannot find `H5Cpp.h`, add the appropriate include path with `-I`. If the linker cannot find the HDF5 libraries, add the appropriate library path with `-L`.

You can check whether HDF5 is available on your system with either

```
h5c++ -show
```

or

```
h5ls --version
```

Chapter 7

Making Plots: DRAGONGAZE (Alpha)

DRAGON Graphical Analysis & Zonal Exploration (DRAGONGAZE) is DRAGON's Python plotting toolkit: a set of standalone scripts that turn the HDF5 frames written by DRAGON (Chapter 5) into PNG images, using Matplotlib. Nothing here needs to be compiled or linked against DRAGON – `DRAGONGAZE/` is a Python package, so each script is run as a module from the command line, from the directory that *contains* `DRAGONGAZE/`, once a simulation has produced at least one output frame.

7.1 Setup

DRAGONGAZE needs **h5py** and **Matplotlib** installed (Section 1.3). Before running any script, check two settings at the top of `Config.py`: `hdf_dir` must point to the same directory the simulation is writing output to (i.e. DRAGONHOARD's own `output_dir`, Section 5.1), and `img_dir` is where the generated plots will be saved. Be sure to create this directory first if it doesn't already exist.

7.2 Generating Plots

Every command below is run from the directory that contains `DRAGONGAZE/` (e.g. the DRAGON repository root), not from inside it – see Section 7.4 for why.

- **1D runs:** run `python -m DRAGONGAZE.GAZE1D` for hydrodynamics, or `python -m DRAGONGAZE.GAZE1DMHD` for MHD. No arguments are needed – each produces one multi-panel image per existing frame, combining density, pressure, energy, and velocity (plus, for MHD, the transverse magnetic field components) into a single figure.
- **2D/3D runs:** list the fields to plot on the command line, e.g. `python -m DRAGONGAZE.GAZE2D rho p Bmag` or `python -m DRAGONGAZE.GAZE3D rho Bmag`. Each named field gets its own image per frame. Available field names are `rho`, `vx`, `vy`, `vz`, `p`, `E`, and (MHD builds) `Bx`, `By`, `Bz`, plus `Bmag` for the field strength $|\mathbf{B}|$ – this is computed automatically, so there's no need to request the components separately just to see it. In addition to the above fluid parameters, any passive scalar can also be plotted by its key name.
- **3D slices:** a field can optionally be suffixed with an axis pair to pick which mid-plane slice to render, e.g. `rho-xy`, `p-xz`, `Bmag-yz` (order doesn't matter – `xy` and `yx` give the same slice). A field with no suffix produces all three mid-plane slices. A field can be listed twice to select two of the three possible slices, for example `rho-xy rho-xz`.

- **Bivariate plots:** to plot two variables as a bivariate heatmap, run `python -m DRAGONGAZE.GAZE2DBi` or `python -m DRAGONGAZE.GAZE3DBi`. These scripts take two and three arguments respectively: the field to be plotted in red, the field to be plotted in blue, and, in 3D, the axis designation (`xy`, `xz`, or `yz`). When both fields are present, the bivariate heatmap will display as magenta, while an absence/minimum of both will be white.

Images are written into `img_dir`, named by field, frame number, and (for 3D) axis pair (e.g. `density_frame_00003.png`, or `density_xy_frame_00003.png` for a 3D slice). A sequence of frames for one field sorts naturally and can be stitched directly into an animation (e.g. with `ffmpeg` or `magick`). To keep such an animation from flickering, DRAGONGAZE fixes each field's colour and axis limits once across every existing frame before it starts plotting, rather than rescaling them frame by frame.

7.3 Customizing Plot Appearance

Everything about how a plot looks is controlled from `Config.py` – the plotting scripts themselves never need to be touched:

- `titles` and `labels` set each field's plot title and axis label – labels support LaTeX-style math via Matplotlib's mathtext, so a density label can be written to render as ρ rather than plain text.
- `cmaps` sets the colormap used for each field; any colormap name from Matplotlib's reference can be used.
- `log_plots` switches a field between a linear and a logarithmic scale.
- `x_mode`/`y_mode`/`z_mode` choose where each axis's origin is labelled from – centred, or at one edge. This is purely cosmetic and never changes the underlying data.
- `plot_title` sets a title shared across every plot, and `time_unit` appends a unit label after the timestamp shown on each plot. DRAGON is unitless by default, so this is left blank unless a specific run has been given physical units.

When plotting passive scalars, you can add the scalar's key to the above dictionaries to control how those plots appear. If no key is provided, DRAGONGAZE will provide default values, and the colormap will be given by `cmaps["default"]`.

7.4 Additional Information for Developers

`HDF5_keys.py` maps each plottable field name to the HDF5 dataset path DRAGONHOARD wrote it under (Section 5.1). These paths are hard-copied from `HDF5_Attrs.hpp` rather than shared with the C++ side in any way, so a key renamed or added there has no effect until `HDF5_keys.py` is updated to match. The failure mode is a plain `KeyError` at plot time, not a build error.

`FileUtils.readField(n, field)` is what actually resolves a field name to an array: for every field except `Bmag` this is a direct lookup via `HDF5_keys.keys`, while `Bmag` is synthesized on the fly as $\sqrt{B_x^2 + B_y^2 + B_z^2}$ since it isn't stored directly. You can use the same pattern to (checking for a special-cased name before falling back to the `keys` lookup) is the place to add any other derived quantity (such as kinetic energy). Be sure to also add entries for the your new field in `Config.py`.

Chapter 8

Validation and Test Cases

DRAGON's verification scheme consists of three components:

1. An extensive suite of unit tests (`Testing/`) that exercises individual components in isolation
2. Additional unit tests which exercise fundamental properties (e.g. conservation laws) of the Godunov scheme on a small grid over a short duration
3. A set of example problems (`Examples/`) that exercise the whole solver against known solutions to standard benchmark problems.

The first two components are typically run together via the `DRAGON_TESTS` target described in Chapter 6. As a rule of thumb, developers should aim to run (and pass) the unit test suite before each commit.

8.1 Unit Test Suite

The main function in `Testing.cpp` runs each component's top-level `verify_*()` entry point, which in turn runs all of the narrower tests for that component and prints a confirmation once each group passes. A complete lists of tests is declared in `Testing.hpp`.

Individual tests are plain `assert`-based checks (via `<cassert>`). In addition to declaring all tests, `Testing.hpp` provides helpers comparing floating-point results: `approx(a, b, rel, abs)` for scalars, and `expect_close(...)` for `vec3`, `PrimitiveState`, and `ConservativeState` objects. These functions accept both a relative and an absolute tolerance, defaulting to `1e-15` and `1e-18` respectively.

8.1.1 Component Tests

In the order of execution, the following sets of component tests can be found in `Testing/Core`

- `FluidElement_Tests.cpp` exercises fluid arithmetic, state conversions, and other properties of the data types described in Section 1.2.1.
- `Grid_Tests.cpp` exercises array construction and indexing described in Section 1.2.2.
- `Boundary/*` exercises the filling of ghost cells by boundary objects (Section 2.3). The file `Boundary/Boundary_Tests.cpp` exercises boundary construction, boundary composition, and the no-op `Boundary::Ignore` class. All other built-in boundary types are tested within their own respective cpp file, with an additional file to test the gated variant of the outflow boundary.

- `IO_Tests.cpp` Exercises the output and restart capabilities of DRAGONHOARD (Chapter 5). The primary tests here consist of writing a grid to a file and then loading it back in; the test passes if the imported grid matches the original. There are also two tests which check that DRAGONHOARD throws when the dimension or size of the constructed grid doesn't match the provided grid.
- `Riemann/*` exercises the Riemann solvers (Section 2.7).
- `TVD_Tests.cpp` exercises MUSCL reconstruction (Section 2.6) and all of the limiters therein.
- `CFL_Tests.cpp` exercises the computation of the CFL timestep bound (Section 2.5).
- `MHD/Faraday_Tests.cpp` exercises portions of the constrained transport machinery (Section 2.9), though the computation of electric fields is not tested explicitly in this version. One of the more important tests here is `verify_ct_divergence_2D/3D()`, which checks that the constrained transport preserves $\nabla \cdot \mathbf{B} = 0$ to machine precision.

8.1.2 Godunov Tests

Rather than exercising individual components, the tests in `Godunov/*` and `MHD/CT_Tests.cpp` exercise the fundamental properties of the Godunov scheme. For a typical test, a small grid is initialised and advanced for some short time interval, after which a series of assertions are run on the output. Notable tests include

- **Periodic Conservation**: a flow within a periodic grid will obey conservation laws for mass, energy, and momentum. This test is particularly good at catching bugs.
- **Domain Decomposition**: the domain-decomposed grid (Section 4.3) will produce the same result as an equivalent monolithic grid. This test also ensures that a sufficient number of ghost cells are allocated by the grid constructor, since an insufficient number of ghosts will lead to subgrids computing different fluxes at their interfaces.
- **Zero Time**: a timestep of zero (`grid.advance(0)`) produces no change
- **1D Match**: For multidimensional flows, a 1D problem can be reproduced exactly over a single timestep.

8.2 Example Problems

The example problems in `Examples/` (Section 6.2) validate DRAGON at the level of complete simulations rather than individual components. This is typically done via comparison against a known solution or published reference result, or through an internal convergence study.

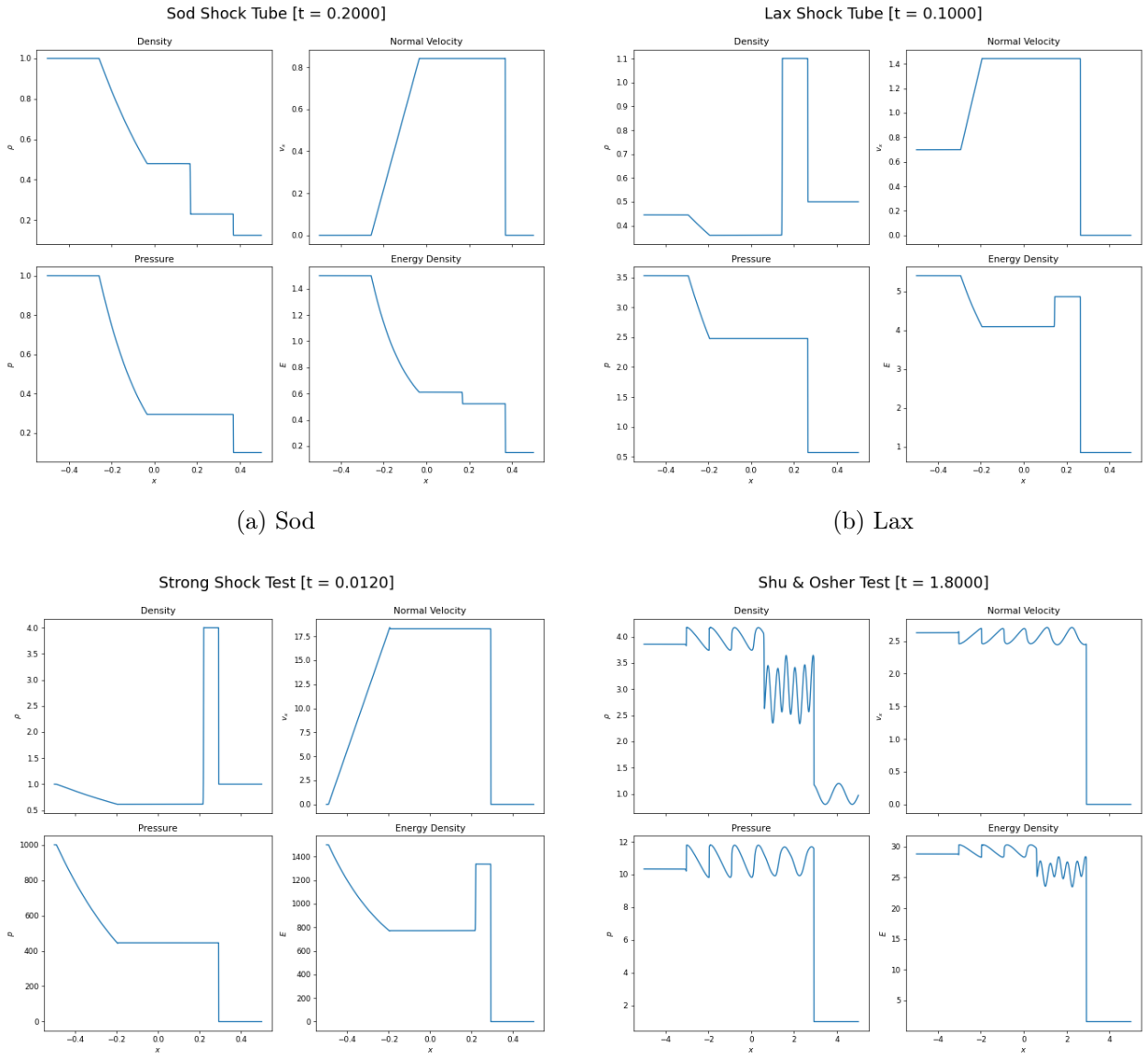
For problems with a convergence study, the example is repeated at successive grid resolutions $n, 2n, 4n, \ldots$. At each resolution, `Problem::problemComplete(...)` computes the error (typically found by comparing the final frame to the initial frame, or to an analytic solution) and records it in the L_1, L_2, and L_∞ (max) norms. The observed convergence rate between two successive resolutions is then

$$q = \log_2(E_n / E_{2n}), \tag{8.1}$$

where E_n is the error at resolution n; for a scheme with formally pth-order accuracy, q should approach p as n grows; the MUSCL-Hancock scheme (Section 2.6) DRAGON uses by default is

formally second order, so $q \to 2$ is the expectation in a well-resolved smooth-flow test. In practice, edge cases in MUSCL's formal order tend to result in the observed order falling somewhere between 1.0 and 2.0.

8.2.1 Examples in One Dimension



(a) Sod

(b) Lax

(c) Strong Shock

(d) Shu & Osher

Figure 8.1: Reference output for the pure-hydrodynamic 1D examples.

The 1D examples (`Examples/1D/`) are classic shock-tube and wave-interaction Riemann problems with well-established reference solutions: Sod (1978), Lax (1954), Shu and Osher (1989), and the Strong Shock test for pure hydrodynamics; and Brio and Wu (1988), citetDaiWoodward1994, and Ryu and Jones (1995) for MHD. Each is a self-contained `Config.h`/`Problem.cpp` pair (plus, for Brio & Wu, a `Constants.h` setting $\gamma = 2$). All of these problems use outflow boundaries, HLLC (hydro) or

Table 8.1: Initial states for the 1D example problems.

Example	MHD	Left state	Both states	Right state
Sod	No	$\rho = 1.0$, $p = 1.0$	$v = 0$	$\rho = 0.125$, $p = 0.1$
Lax	No	$\rho = 0.445$, $v_x = 0.698$, $p = 3.528$	$v_y = v_z = 0$	$\rho = 0.5$, $v_x = 0$, $p = 0.571$
Strong Shock	No	$p = 1000.0$	$\rho = 1.0$, $v = 0$	$p = 0.01$
Shu & Osher	No	$\rho = 3.857143$, $v_x = 2.629369$, $p = 10.333333$	$v_y = v_z = 0$	$\rho = 1.0 + 0.2\sin(5x)$, $v_x = 0$, $p = 1.0$
Brio & Wu	Yes	$\rho = 1.0$, $p = 1.0$, $B_y = \sqrt{4\pi}$	$v = 0$, $B_x = 0.75\sqrt{4\pi}$, $B_z = 0$	$\rho = 0.125$, $p = 0.1$, $B_y = -\sqrt{4\pi}$
Dai & Woodward	Yes	$\rho = 1.08$, $v = (1.2, 0.01, 0.5)$, $p = 0.95$, $B_y = 3.6$	$B_x = 2.0$, $B_z = 2.0$	$\rho = 1.0$, $v = (0, 0, 0)$, $p = 1.0$, $B_y = 4.0$
Ryu & Jones	Yes	$v_x = 10.0$, $p = 20.0$	$\rho = 1.0$, $v_y = v_z = 0$, $B = (5, 5, 0)$	$v_x = -10.0$, $p = 1.0$

HLLD (MHD) as the Riemann solver, and MC limiting for hydro / MINMOD for MHD – MC was found to produce spurious oscillations near strong shocks in the MHD cases. The specific inputs for each test are listed in Table 8.1. Figure 8.1 shows DRAGON's output for the hydrodynamic problems, generated with DRAGONGAZE (Chapter 7); the corresponding plots for the MHD problems can be found in Appendix A. Table 8.1 lists the initial left, right, and shared states set up in each problem's `Problem.cpp`.

8.2.2 Hydrodynamic Examples in Two and Three dimensions

The multidimensional hydrodynamic examples (`Examples/Hydro/`) move past 1D shock tubes to exercise genuinely multidimensional behaviour.For these examples it is of particular importance to exercise the CTU integration (Section 2.6.4), since the hydrodynamic 12-solve code path is distinct from MHD's 6-solve algorithm. Each example provides a `Problem2D.cpp`; some tests also provide a `Problem3D.cpp` so the same physical setup can be run in either dimensionality.

Diagonal Advection Test

Diagonal Advection is DRAGON's simplest convergence test: a smooth sinusoidal density perturbation, $\rho(x, y, 0) = \rho_0 + \rho_1 \sin\big(2\pi(x+y)\big)$, is advected at unit velocity along the grid diagonal ($\mathbf{v} = (1, 1)$ in 2D, $(1, 1, 1)$ in 3D). Because the flow is smooth, periodic, and uniform-velocity advection, we can compare the computed solution to the exact analytic solution

$$\rho_{\text{exact}}(x, y, t) = \rho_0 + \rho_1 \sin\big(2\pi(x + y - 2t)\big) \tag{8.2}$$

in 2D (and analogously with a $3t$ phase shift in 3D). We have run this problem using HLLC (Section 2.7) paired with three different limiters (Section 2.6) in 2D, plus one 3D run. The results are presented in Tables 8.2–8.5.

The data is broadly consistent with the second-order expectation: L_1 converges at $q \approx 1.9$–2.0 for the two smoother limiters (MC, minmod), L_2 and L_∞ converge more slowly since they weight the (still slightly under-resolved) peak of the sine wave more heavily. Superbee meanwhile converges somewhat worse than the other limiters. This is expected, as it is the most compressive of the three limiters, and for a smooth-flow test there is no shock to sharpen so it only adds unnecessary numerical dissipation.

Table 8.2: Diagonal Advection (2D) convergence, HLLC + Minmod.

n	L_1 error	L_1 rate	L_2 error	L_2 rate	L_∞ error	L_∞ rate
128	1.034×10^{-3}	–	1.412×10^{-3}	–	3.703×10^{-3}	–
256	2.863×10^{-4}	1.85	4.494×10^{-4}	1.65	1.512×10^{-3}	1.29
512	7.834×10^{-5}	1.87	1.422×10^{-4}	1.66	6.103×10^{-4}	1.31

Table 8.3: Diagonal Advection (2D) convergence, HLLC + MC.

n	L_1 error	L_1 rate	L_2 error	L_2 rate	L_∞ error	L_∞ rate
128	2.249×10^{-4}	–	3.107×10^{-4}	–	9.426×10^{-4}	–
256	5.631×10^{-5}	2.00	9.315×10^{-5}	1.74	3.880×10^{-4}	1.28
512	1.407×10^{-5}	2.00	2.812×10^{-5}	1.73	1.593×10^{-4}	1.28

Table 8.4: Diagonal Advection (2D) convergence, HLLC + Superbee.

n	L_1 error	L_1 rate	L_2 error	L_2 rate	L_∞ error	L_∞ rate
128	6.932×10^{-4}	–	9.097×10^{-4}	–	2.784×10^{-3}	–
256	1.987×10^{-4}	1.80	2.922×10^{-4}	1.64	1.272×10^{-3}	1.13
512	5.302×10^{-5}	1.90	9.130×10^{-5}	1.68	5.151×10^{-4}	1.30

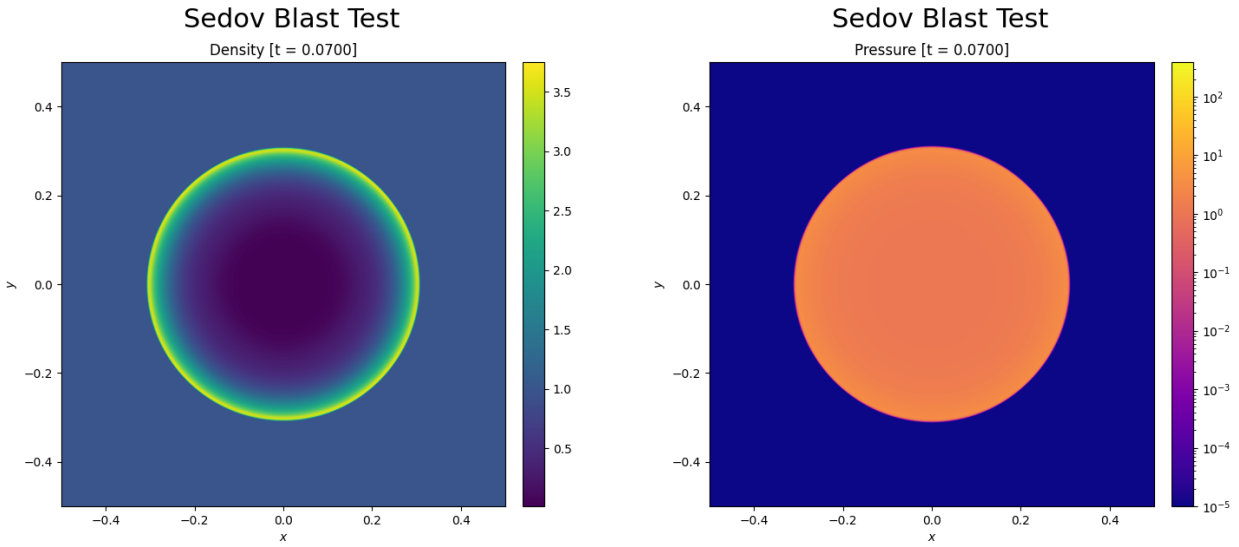
Table 8.5: Diagonal Advection (3D) convergence, HLLC + MC.

n	L_1 error	L_1 rate	L_2 error	L_2 rate	L_∞ error	L_∞ rate
64	1.168×10^{-3}	–	1.390×10^{-3}	–	2.910×10^{-3}	–
128	3.215×10^{-4}	1.86	4.245×10^{-4}	1.71	1.219×10^{-3}	1.25
256	8.195×10^{-5}	1.97	1.283×10^{-4}	1.73	5.142×10^{-4}	1.25

Sedov Blast Test

The Sedov Blast test deposits a fixed amount of energy E_{blast} into a small region of radius r_0 (a handful of cells wide) at the centre of an otherwise uniform, low-pressure ambient medium. The resulting point explosion is then allowed to expand outward. Concretely, this is implemented as a pressure spike: `Problem::makeProblem()` first counts how many cells actually fall within r_0 of the origin at the chosen resolution, then scales the per-cell blast pressure so that the total deposited energy is (to grid resolution) independent of n. `Problem::initialFluidState(...)` then applies that pressure only within r_0, leaving density uniform ($\rho = 1$) and the exterior at the low ambient pressure $p_{\text{amb}} = 10^{-5}$ everywhere else.

The expected result of this problem is the self-similar Sedov–Taylor blast wave (Taylor, 1950; Sedov, 1959): a thin, strong spherical (circular, in 2D) shock expanding outward, enclosing a hot, nearly-uniform-pressure interior and leaving vacuum-like ambient conditions untouched ahead of the shock front. However, such a strong, grid-aligned shock is prone to the carbuncle instability (Quirk, 1994), a well-known failure mode in which a contact-preserving solver like HLLC lets a spurious, grid-aligned perturbation grow along the shock front. To avoid this, we use the HLLE (Section 2.7.3) Riemann solver rather than HLLC for this example. HLLE has no separate contact wave, so it is more diffusive across contacts; that extra diffusion is exactly what damps the carbuncle-forming perturbation, at the cost of a somewhat more smeared shock front.



(a) Density

(b) Pressure (log scale)

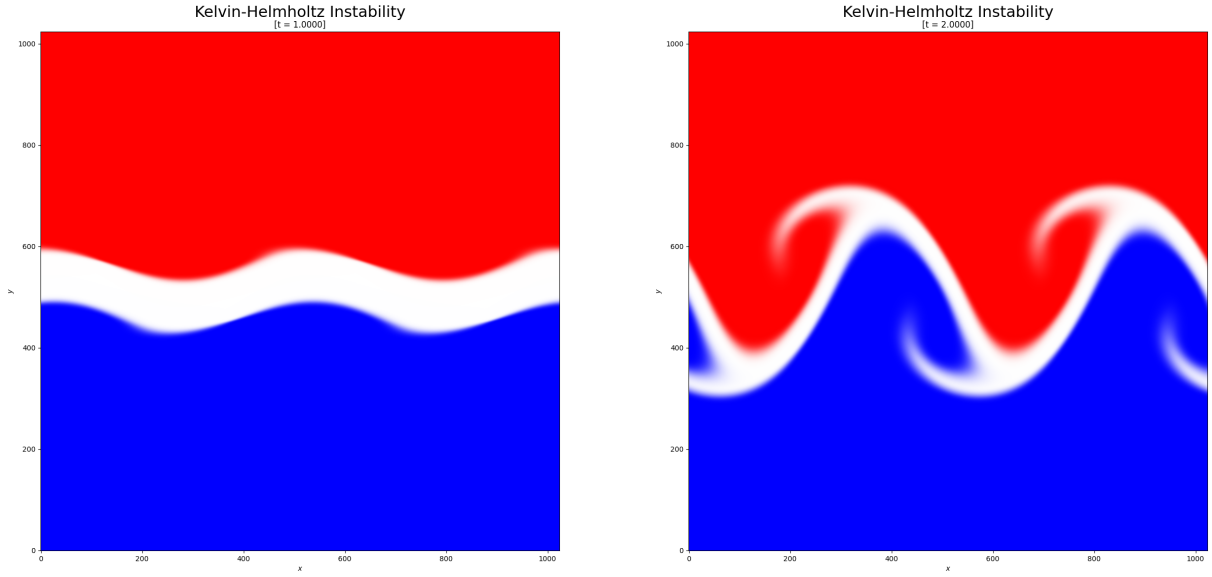
Figure 8.2: Reference output for the 2D Sedov Blast test at $t = 0.07$.

The results plotted in Figure 8.2 are as expected: the density panel shows the characteristic thin high-density shell trailing the shock front with a nearly-evacuated interior, and the (log-scaled) pressure panel shows the sharp several-orders-of-magnitude jump across the shock separating the hot post-shock bubble from the undisturbed ambient medium.

Kelvin–Helmholtz Instability

The Kelvin–Helmholtz example sets up a shear layer, following the well-posed construction of McNally et al. (2012): a denser right moving band ($\rho_1 = 2$, $v_x = +V_0$) separated from a lighter left-

moving ambient medium ($\rho_2 = 1, v_x = -V_0$) by two tanh-smoothed interfaces. A small, spatially-localized transverse velocity perturbation is seeded at both interfaces – a $\sin(2\pi\,\texttt{kmode}\,x)$ pattern windowed by a Gaussian centred on each interface; this gives the shear instability something to grow from rather than waiting on grid noise. Two passive scalars are used to trace the fluid: one is initialised to 1 in the upper ambient region and 0 everywhere else, and the other likewise in the lower ambient region.



(a) $t = 1.0$ (b) $t = 2.0$

Figure 8.3: A heatmap of two tracers, showing the growth of the Kelvin–Helmholtz instability.

Unlike the other two examples in this section, Kelvin–Helmholtz has no closed-form solution to compare against – growth of the shear instability is chaotic at late times, so this example is checked qualitatively, by confirming the characteristic vortex roll-up pattern appears and grows as expected. Figure 8.3 shows this development: at $t = 1$ the perturbation is still a small-amplitude wave on the shear interface, while by $t = 2$ it has rolled up into the pair of vortices expected from `kmode=2` (one every half-period of the seeded perturbation), with visible secondary structure at the vortex cores from the instability continuing to cascade to smaller scales.

8.2.3 MHD Examples in Two and Three dimensions

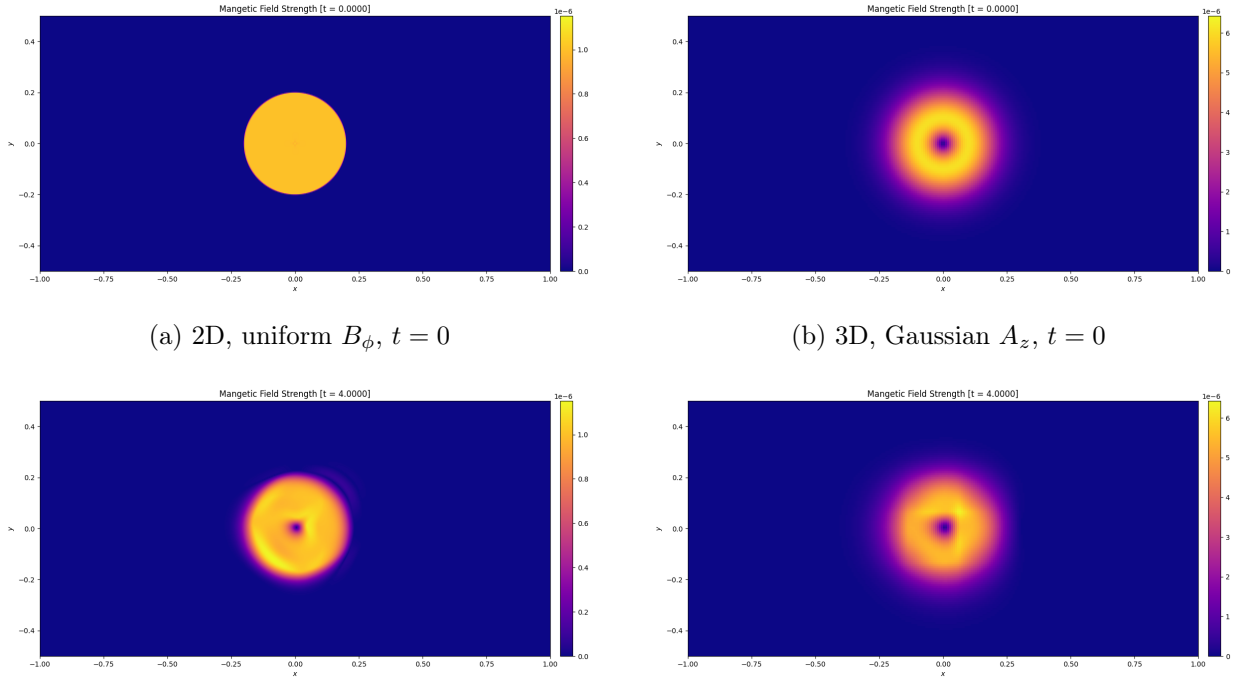
Field Loop Advection

The field loop advection test (Gardiner and Stone, 2005, 2008) carries a weak magnetic field loop across a periodic domain at uniform velocity. The field is deliberately weak ($B_0 = 10^{-6}$) in order to isolate inductive advection from the effects of magnetic pressure and tension altering the fluid. Any distortion, diffusion, or divergence error in the transported field can only come from the induction step itself. The background velocity is oblique to all grid axes: $(v_x, v_y) = (2, 0.5)$ in 2D and $(v_x, v_y, v_z) = (2, 1, 1)$ in 3D (with the loop uniform in the z direction).

The example problem supports two loop profiles. The *uniform* profile sets the vector potential $A_z = B_0 \max(r_0 - r, 0)$, a cone that is linear in r and therefore gives a field of constant magnitude $|B_\phi| = B_0$ everywhere inside $r < r_0$, dropping to zero outside. The *Gaussian* profile instead sets

$A_z = B_0 \exp\big(-\frac{1}{2}(r/r_0)^2\big)$, giving a smooth field magnitude that peaks in an annulus around the loop centre and has no kink.

Because the domain is periodic and the advection velocity is uniform, the chosen run time is an integer number of periods along all axes, so the field should return exactly to its initial shape. Figure 8.4 shows this for both profiles, both at the initial and final times.



(a) 2D, uniform B_ϕ, $t = 0$

(b) 3D, Gaussian A_z, $t = 0$

(c) 2D, uniform B_ϕ, $t = 4$

(d) 3D, Gaussian A_z, $t = 4$

Figure 8.4: Magnetic field strength for the Field Loop Advection test, before and after two full advection periods.

As with the other periodic advection tests, `Problem::problemComplete(...)` validates the run by reloading the initial frame from disk with `DRAGONHOARD::loadFromFile(...)` and differencing it against the final state, componentwise in $\mathbf{B}$. The reported L_1 error is $|\overline{\delta\mathbf{B}}|$, the magnitude of the mean absolute error vector. Convergence data for both profiles are given in Table 8.6–Table 8.7. The $n = 512$ cases were not run in 3D on account of the diminishing returns for another high-resolution simulation being insufficient to justify the multi-day runtime.

Table 8.6: Field Loop Advection, uniform B_ϕ convergence.

	2D		3D	
n	L_1 error	L_1 rate	L_1 error	L_1 rate
64	5.211×10^{-8}	–	5.657×10^{-8}	–
128	2.718×10^{-8}	0.94	3.040×10^{-8}	0.90
256	1.615×10^{-8}	0.74	1.771×10^{-8}	0.78
512	1.052×10^{-8}	0.62	–	–

Table 8.7: Field Loop Advection, Gaussian A_z convergence.

	2D		3D	
n	L_1 error	L_1 rate	L_1 error	L_1 rate
64	2.088×10^{-7}	–	2.309×10^{-7}	–
128	8.032×10^{-8}	1.38	9.191×10^{-8}	1.32
256	2.702×10^{-8}	1.57	3.214×10^{-8}	1.51
512	9.061×10^{-9}	1.57	–	–

Unlike Diagonal Advection (Section 8.2.2), which converges at $q \approx 1.9$–2.0, the loop advection error rates sit well below second order: $q \approx 0.6$–0.9 for the uniform-B_ϕ profile and $q \approx 1.3$–1.6 for the Gaussian profile. For the uniform-B_ϕ profile, this is most likely the result of the discontinuities at $r = 0$ and r_0: a scheme built around smooth reconstruction cannot converge faster than first order through a kink like this, which caps its observed rate close to 1. The Gaussian profile has no such kink and converges noticeably faster, though it is still below the $q \approx 2$ seen for Diagonal Advection.

Linear Wave Convergence

The MHD linear wave test (Gardiner and Stone, 2008) seeds a single, exactly linear eigenmode of the MHD equations on top of a uniform background state and lets it propagate. The background is oriented obliquely to the grid so that the wave crosses all three axes rather than propagating along a single grid line, allowing this problem to properly test the convergence of DRAGON's CTU-CT implementation.

All seven wave modes are implemented: fast, Alfvén, slow, and entropy, with + and − variants for the first three. The background is perturbed by $\epsilon R \cos(2\pi x_1)$, where R is the analytic eigenvector for the selected mode, x_1 is the rotated propagation coordinate, and $\epsilon = 10^{-6}$ is small enough to stay in the linear regime.

As with Field Loop Advection, validation reloads the initial frame with `DRAGONHOARD::loadFromFile(...)` and differences it against the state after the wave has propagated an integer number of wavelengths back to its starting position. Here the comparison spans every conserved field at once: the componentwise L_1 errors in ρ, $\rho\mathbf{v}$, $\mathbf{B}$, and E are all computed, and then combined into a single scalar

$$L_1 = \sqrt{L_1(\rho)^2 + L_1(\rho\mathbf{v})^2 + L_1(\mathbf{B})^2 + L_1(E)^2}. \tag{8.3}$$

Tables 8.8–8.11 give the results for both propagation directions (+/−) of the fast, Alfvén, and slow modes, as well as for the (direction-independent) entropy mode.

Table 8.8: MHD Linear Wave (3D, fast mode) convergence.

n	(+) L_1 error	(+) L_1 rate	(−) L_1 error	(−) L_1 rate
16	5.389×10^{-7}	–	5.389×10^{-7}	–
32	1.171×10^{-7}	2.20	1.175×10^{-7}	2.20
64	3.466×10^{-8}	1.76	3.463×10^{-8}	1.76

The fast, Alfvén, and slow modes all converge close to second order at these resolutions, as expected for smooth linear waves on the (formally second-order) MUSCL–Hancock scheme. The

Table 8.9: MHD Linear Wave (3D, Alfvén mode) convergence.

n	(+) L_1 error	(+) L_1 rate	(−) L_1 error	(−) L_1 rate
16	4.657×10^{-7}	–	4.657×10^{-7}	–
32	9.576×10^{-8}	2.28	9.573×10^{-8}	2.28
64	2.481×10^{-8}	1.95	2.482×10^{-8}	1.95

Table 8.10: MHD Linear Wave (3D, slow mode) convergence.

n	(+) L_1 error	(+) L_1 rate	(−) L_1 error	(−) L_1 rate
16	1.723×10^{-7}	–	1.723×10^{-7}	–
32	4.457×10^{-8}	1.95	4.457×10^{-8}	1.95
64	1.321×10^{-8}	1.75	1.322×10^{-8}	1.75

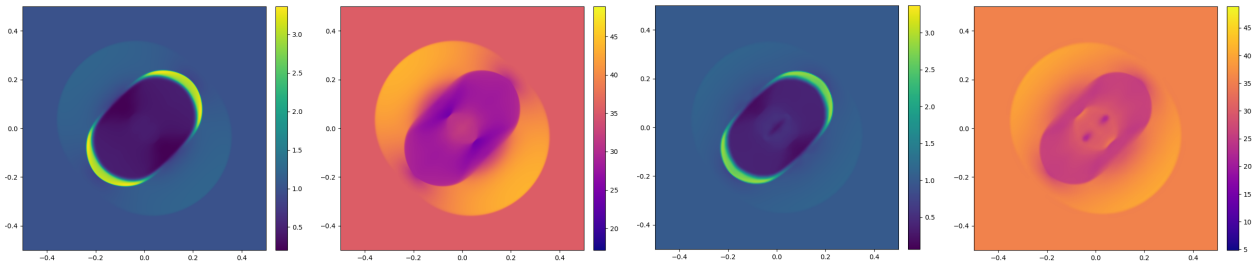
Table 8.11: MHD Linear Wave (3D, entropy mode) convergence.

n	L_1 error	L_1 rate
16	1.250×10^{-7}	–
32	4.511×10^{-8}	1.47
64	1.381×10^{-8}	1.71

entropy mode converges somewhat more slowly; this mode is purely advected, so its accuracy is governed entirely by the scalar advection error similar in character to the sub-second-order behaviour seen for the piecewise-linear loop profile in Section 8.2.3. For the fast, Alfvén, and slow modes, the + and − branches agree with each other to within a few percent, as expected by symmetry.

MHD Blast

The MHD Blast test (Gardiner and Stone, 2008) is the magnetized counterpart to the hydrodynamic Sedov Blast (Section 8.2.2): a small central region of radius r_0 is set to a much higher pressure ($p_{\rm blast} = 100$) than the surrounding ambient medium ($p_{\rm amb} = 1$), with uniform density and zero velocity everywhere. Unlike the hydro version, the domain is threaded by a strong, uniform background magnetic field, with magnetic pressure comparable to or larger than the blast overpressure. This field is oriented diagonally at 45° to the grid by default to better exercise the CTU machinery.



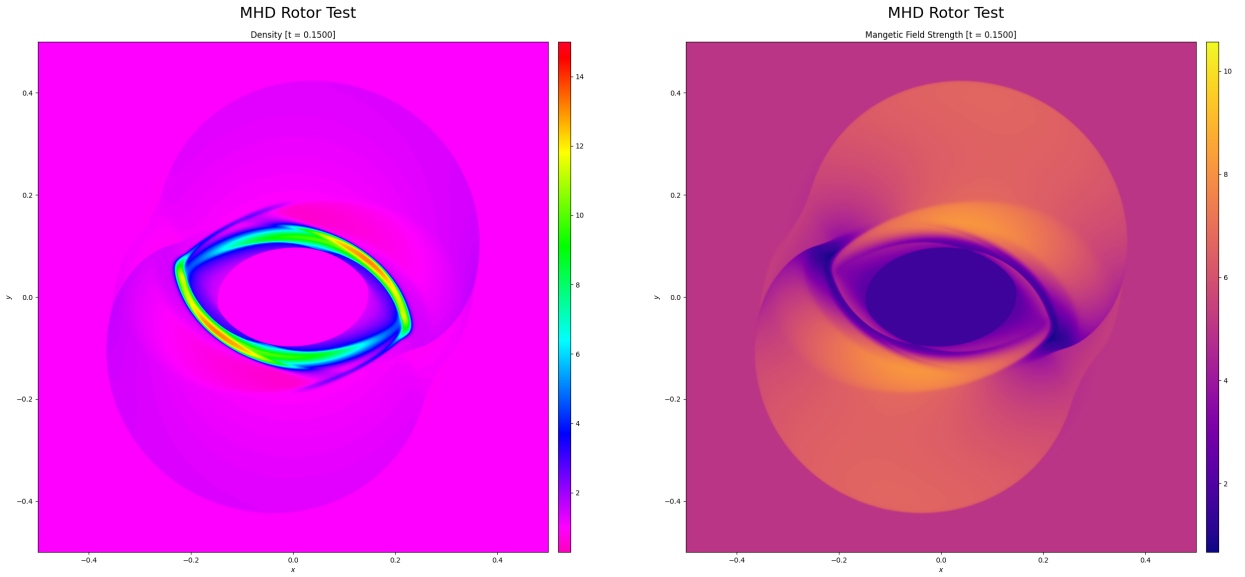
(a) 2D, Density (b) 2D, $|\mathbf{B}|$ (c) 3D, Density (d) 3D, $|\mathbf{B}|$

Figure 8.5: MHD Blast test at $t = 0.02$, 2D and 3D (mid-plane slices).

Figure 8.5 shows the result in both 2D and 3D. As a result of magnetic tension along the field lines, the blast expansion is no longer circular as was the case in the hydrodynamic counterpart. Instead, the anisotropic blast expansion should demonstrate the shock moving faster along the field lines than in other directions. The magnetic field strength panels show evacuated cavities near the blast centre, where the explosion has pushed the field outward and compressed it into the bright rim.

MHD Rotor

The MHD Rotor test (Balsara and Spicer, 1999; Tóth, 2000) embeds a dense, rigidly rotating disk ($\rho_0 = 10$, angular velocity $\omega = 20$, radius $r_0 = 0.1$) in a lighter, stationary ambient medium ($\rho_{\text{amb}} = 1$), with a linear taper in density and velocity between r_0 and $r_1 = 0.115$ so the initial condition is continuous. A uniform background field $B_x = B_0$ threads the whole domain. Though the problem has no closed analytical solution, it is still useful in demonstrating expected behaviours of MHD: as the disk spins up the surrounding fluid, it winds the field lines into a strong torsional structure, launching Alfvén waves outward and progressively braking the rotor , thus testing the coupling between the magnetic tension force and rotational flow.



(a) Density (b) $|\mathbf{B}|$

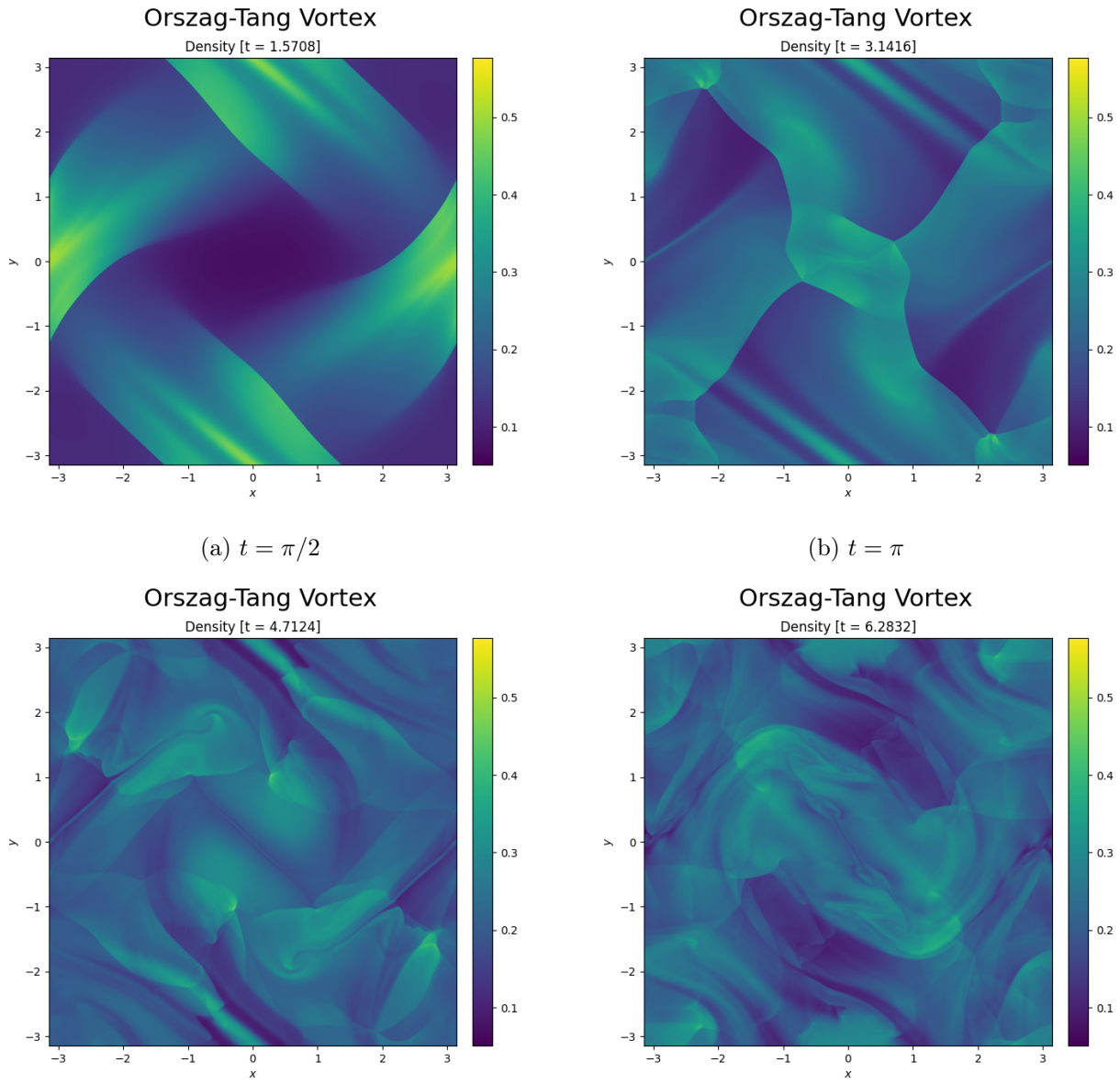
Figure 8.6: MHD Rotor test at $t = 0.15$.

Figure 8.6 shows the expected result: the dense rotor has been drawn out into a thin, sheared ring bounded by a pair of spiral shocks, while the magnetic field strength panel shows the low-field cavity carved out at the centre of the rotor and the wound-up, compressed field trailing behind the spiral arms – the signature of the torsional Alfvén waves the rotation launches into the ambient medium.

Orszag–Tang

The Orszag–Tang vortex (Orszag and Tang, 1979) is a fully periodic problem on $[0, 2\pi]^2$: the velocity field $\mathbf{v} = (-\sin y,\ \sin x, 0)$ sets up a large-scale vortical flow, while the vector potential

$A_z = B_0\big(\cos y + \frac{1}{4}\cos 2x\big)$ seeds a magnetic field with an X-point current sheet at the origin. As with the MHD Rotor, there is no closed-form solution to compare against; instead this problem is a visual benchmark of a code's ability to handle the development of MHD turbulence, evolving a smooth initial vortex into a cascade of interacting shocks and current sheets.



(a) $t = \pi/2$ (b) $t = \pi$

(c) $t = 3\pi/2$ (d) $t = 2\pi$

Figure 8.7: Density evolution of the Orszag–Tang vortex.

Figure 8.7 shows this development. At $t = \pi/2$, the flow is still close to the smooth initial vortex, with the first diagonal shear bands just beginning to steepen. By $t = \pi$, these have sharpened into well-defined oblique shocks bracketing the central current sheet. Finally, at $t = 3\pi/2$ and $t = 2\pi$ the interacting shocks have broken the flow up into a web of small-scale structure characteristic of MHD turbulence.

Appendix A

Additional Test Problem Plots

Full-page versions of the MHD 1D reference plots referenced from Section 8.2, one per figure so that all six-to-eight panels of each remain legible.

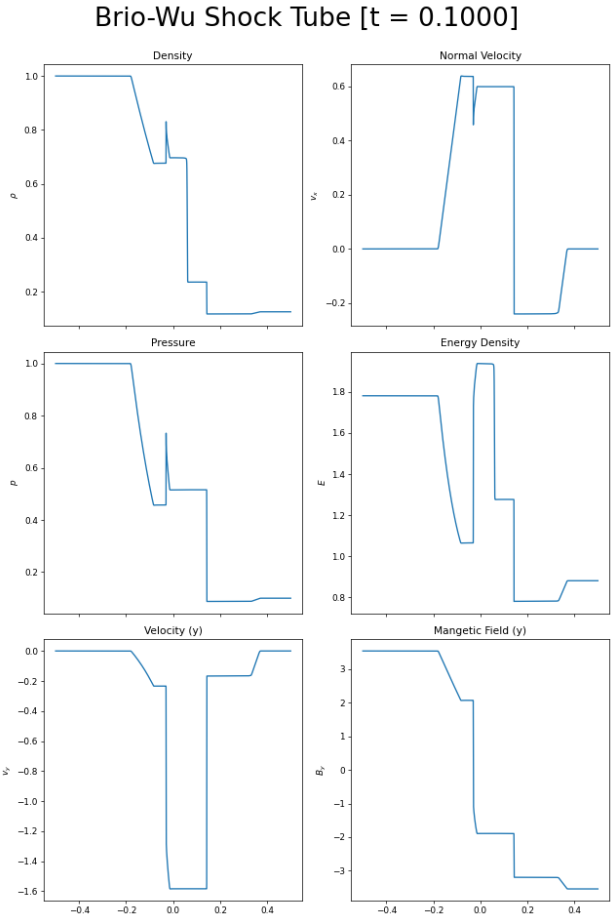


Figure A.1: Reference output for the Brio & Wu shock tube.

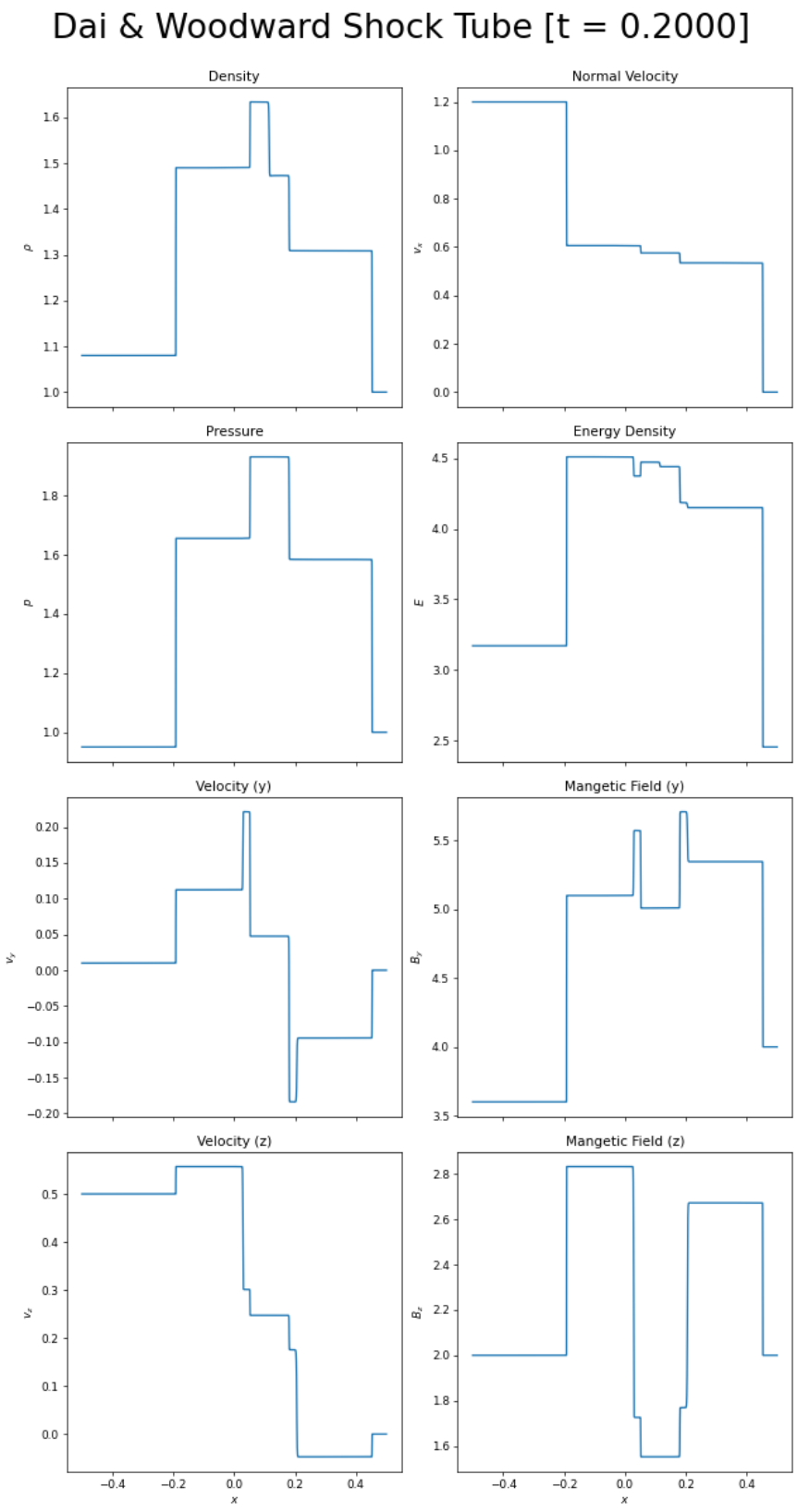


Figure A.2: Reference output for the Dai & Woodward shock tube.

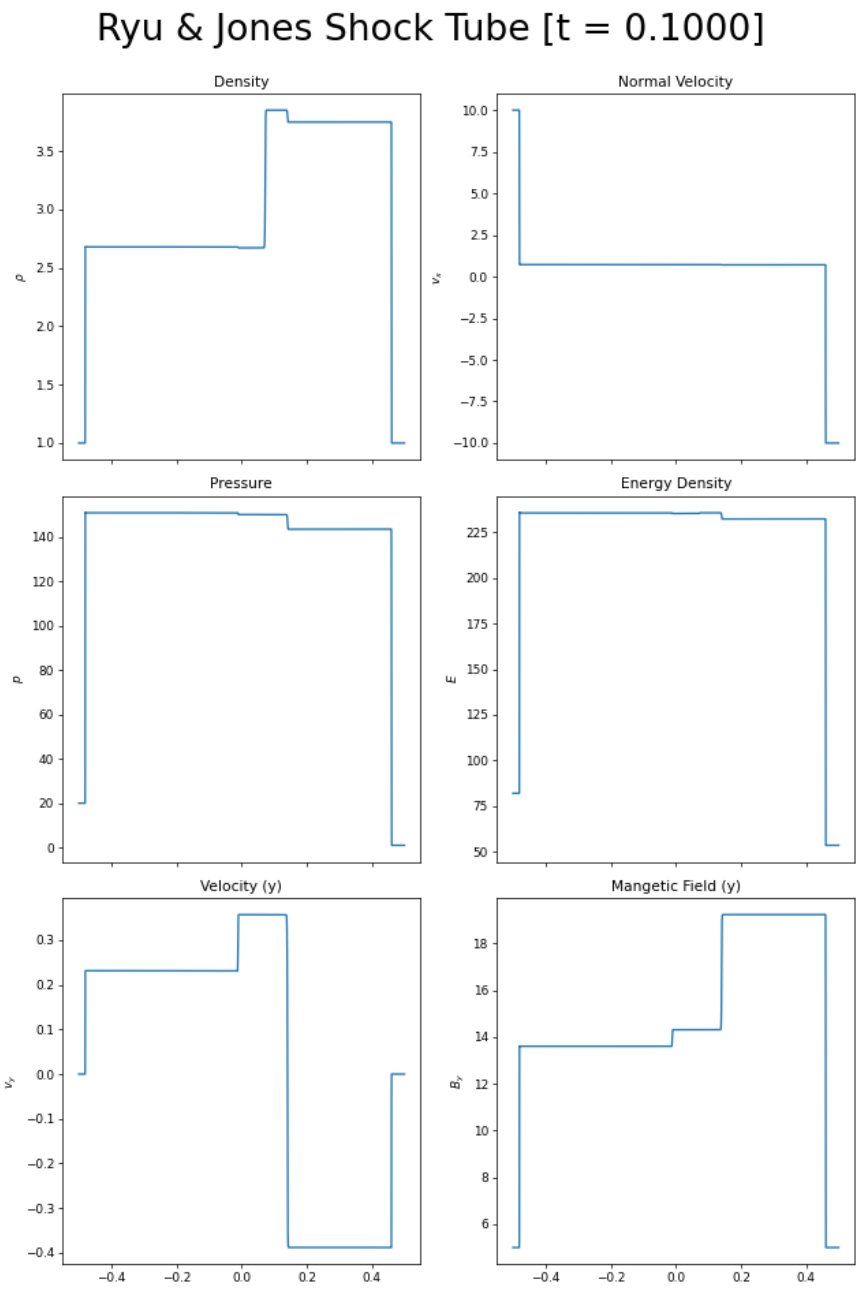


Figure A.3: Reference output for the Ryu & Jones shock tube.

Appendix B

Version History

1 September 2026: DRAGON v1.1 introduced the following changes

- An API for source terms
- The `PrimitiveState += ConservativeState` operator has been rearchitected to improve stability (Section 2.8)

1 September 2026: DRAGON v1.1 introduced the following changes

- Added support for user-defined passive scalars (Section 2.10).
  - Updated Kelvin-Helmholtz example to demonstrate passive scalars as tracers.
  - Added support for plotting any passive scalar in DRAGONGAZE
- Added an atomic counter (Section 4.4) which is used to trigger a step restart if too many fallback behaviours have occurred in one advancement.
- Added new DRAGONGAZE scripts for making bivariate plots.
- Additional unit tests to directly verify DRAGONWING
- Streamlined advancement and exception handling for domain-decomposed grids
  - New `Grid::advance_step(dt)` delegates to `split_step` or `unsplit_step` as appropriate.
  - `launchParallel` now calls `grid.advance_step` instead of `grid.advance`, calls `requestRestart()` directly if an exception is thrown.
  - Removed: `Grid::on_step_fail`, `MULTITHREAD_UNAVAILABLE`

26 August 2026: Initial release of DRAGON v1.0

Bibliography

Dinshaw S. Balsara and Daniel S. Spicer. A staggered mesh algorithm using high order Godunov fluxes to ensure solenoidal magnetic fields in magnetohydrodynamic simulations. *Journal of Computational Physics*, 149(2):270–292, 1999. doi: 10.1006/jcph.1998.6153.

M. Brio and C. C. Wu. An upwind differencing scheme for the equations of ideal magnetohydrodynamics. *Journal of Computational Physics*, 75(2):400–422, 1988. doi: 10.1016/0021-9991(88)90120-9.

Bernd Einfeldt. On Godunov-type methods for gas dynamics. *SIAM Journal on Numerical Analysis*, 25(2):294–318, 1988. doi: 10.1137/0725021.

Charles R. Evans and John F. Hawley. Simulation of magnetohydrodynamic flows: A constrained transport method. *The Astrophysical Journal*, 332:659–677, 1988. doi: 10.1086/166684.

Thomas A. Gardiner and James M. Stone. An unsplit Godunov method for ideal MHD via constrained transport. *Journal of Computational Physics*, 205(2):509–539, 2005. doi: 10.1016/j.jcp.2004.11.016.

Thomas A. Gardiner and James M. Stone. An unsplit Godunov method for ideal MHD via constrained transport in three dimensions. *Journal of Computational Physics*, 227(8):4123–4141, 2008. doi: 10.1016/j.jcp.2007.12.017.

Ami Harten and James M. Hyman. Self adjusting grid methods for one-dimensional hyperbolic conservation laws. *Journal of Computational Physics*, 50(2):235–269, 1983. doi: 10.1016/0021-9991(83)90066-9.

Amiram Harten, Peter D. Lax, and Bram van Leer. On upstream differencing and Godunov-type schemes for hyperbolic conservation laws. *SIAM Review*, 25(1):35–61, 1983. doi: 10.1137/1025002.

Peter D. Lax. Weak solutions of nonlinear hyperbolic equations and their numerical computation. *Communications on Pure and Applied Mathematics*, 7(1):159–193, 1954. doi: 10.1002/cpa.3160070112.

Colin P. McNally, Wladimir Lyra, and Jean-Claude Passy. A well-posed Kelvin–Helmholtz instability test and comparison. *The Astrophysical Journal Supplement Series*, 201(2):18, 2012. doi: 10.1088/0067-0049/201/2/18.

Takahiro Miyoshi and Kanya Kusano. A multi-state HLL approximate Riemann solver for ideal magnetohydrodynamics. *Journal of Computational Physics*, 208(1):315–344, 2005. doi: 10.1016/j.jcp.2005.02.017.

Steven A. Orszag and Cha-Mei Tang. Small-scale structure of two-dimensional magnetohydrodynamic turbulence. *Journal of Fluid Mechanics*, 90(1):129–143, 1979. doi: 10.1017/S002211207900210X.

James J. Quirk. A contribution to the great Riemann solver debate. *International Journal for Numerical Methods in Fluids*, 18(6):555–574, 1994. doi: 10.1002/fld.1650180603.

P. L. Roe. Approximate Riemann solvers, parameter vectors, and difference schemes. *Journal of Computational Physics*, 43(2):357–372, 1981. doi: 10.1016/0021-9991(81)90128-5.

P. L. Roe. Characteristic-based schemes for the Euler equations. *Annual Review of Fluid Mechanics*, 18:337–365, 1986. doi: 10.1146/annurev.fl.18.010186.002005.

P. L. Roe and J. Pike. Efficient construction and utilisation of approximate Riemann solutions. In R. Glowinski and J.-L. Lions, editors, *Computing Methods in Applied Sciences and Engineering, VI*, pages 499–518. North-Holland, Amsterdam, 1984.

Dongsu Ryu and T. W. Jones. Numerical magnetohydrodynamics in astrophysics: Algorithm and tests for one-dimensional flow. *The Astrophysical Journal*, 442:228–258, 1995. doi: 10.1086/175437.

L. I. Sedov. *Similarity and Dimensional Methods in Mechanics*. Academic Press, New York, 1959.

Chi-Wang Shu and Stanley Osher. Efficient implementation of essentially non-oscillatory shock-capturing schemes, II. *Journal of Computational Physics*, 83(1):32–78, 1989. doi: 10.1016/0021-9991(89)90222-2.

Gary A. Sod. A survey of several finite difference methods for systems of nonlinear hyperbolic conservation laws. *Journal of Computational Physics*, 27(1):1–31, 1978. doi: 10.1016/0021-9991(78)90023-2.

Gilbert Strang. On the construction and comparison of difference schemes. *SIAM Journal on Numerical Analysis*, 5(3):506–517, 1968. doi: 10.1137/0705041.

Geoffrey Ingram Taylor. The formation of a blast wave by a very intense explosion. I. theoretical discussion. *Proceedings of the Royal Society of London A*, 201(1065):159–174, 1950. doi: 10.1098/rspa.1950.0049.

The HDF Group. Hierarchical data format, version 5. https://www.hdfgroup.org/solutions/hdf5/, 1997–2026.

E. F. Toro, M. Spruce, and W. Speares. Restoration of the contact surface in the HLL-Riemann solver. *Shock Waves*, 4(1):25–34, 1994. doi: 10.1007/BF01414629.

Eleuterio F. Toro. *Riemann Solvers and Numerical Methods for Fluid Dynamics: A Practical Introduction*. Springer, Berlin, 3rd edition, 2009. doi: 10.1007/b79761.

Gábor Tóth. The $\nabla \cdot \mathbf{B} = 0$ constraint in shock-capturing magnetohydrodynamics codes. *Journal of Computational Physics*, 161(2):605–652, 2000. doi: 10.1006/jcph.2000.6519.

G. D. van Albada, B. van Leer, and W. W. Roberts. A comparative study of computational methods in cosmic gas dynamics. *Astronomy and Astrophysics*, 108:76–84, 1982.

Bram van Leer. Towards the ultimate conservative difference scheme. II. monotonicity and conservation combined in a second-order scheme. *Journal of Computational Physics*, 14(4):361–370, 1974. doi: 10.1016/0021-9991(74)90019-9.

Bram van Leer. Towards the ultimate conservative difference scheme. IV. a new approach to numerical convection. *Journal of Computational Physics*, 23(3):276–299, 1977. doi: 10.1016/0021-9991(77)90095-X.

Bram van Leer. Towards the ultimate conservative difference scheme. V. a second-order sequel to Godunov's method. *Journal of Computational Physics*, 32(1):101–136, 1979. doi: 10.1016/0021-9991(79)90145-1.